

ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



TOÁN ỨNG DỤNG VÀ THỐNG KÊ (MTH00051)

BÁO CÁO ĐỒ ÁN 2

Data Fitting và Phương Pháp OLS

Môn học:	Toán ứng dụng và thống kê	
Giảng viên:	ThS. Võ Nam Thực Đoàn	
	ThS. Lê Nhật Nam	
	TS. Đỗ Đức Hòa	
	ThS. Nguyễn Hữu Toàn	
Sinh viên thực hiện:	Lê Hoàng Hiếu	(MSSV: 24120048)
	Lê Nguyễn Gia Huy	(MSSV: 24120061)
	Trần Nguyễn Anh Khoa	(MSSV: 24120075)
	Nguyễn Hoàng Nam	(MSSV: 24120097)
	Nguyễn Anh Sang	(MSSV: 24120131)
Lớp:	24CTT2	



Mục lục

Danh sách thành viên và Phân công công việc	3
1 PHẦN 1: LÝ THUYẾT DATA FITTING VÀ MINH HỌA	4
1.1 Bài toán Data Fitting	4
1.1.1 Phát biểu bài toán tổng quát:	4
1.1.2 Các Giả Thiết Gauss–Markov	5
1.2 Phương pháp Ordinary Least Squares (OLS)	5
1.2.1 Hàm mất mát và Nghiệm OLS	5
1.2.2 Ma Trận Chiều và Hat Matrix	7
1.2.3 Định Lý Gauss–Markov	8
1.2.4 Ước lượng phương sai nhiễu	9
1.3 Suy diễn thống kê và Đánh giá mô hình	10
1.3.1 Hệ số xác định R^2 và R^2 hiệu chỉnh	10
1.3.2 Kiểm định giả thuyết (t-test, F-test)	10
1.4 Các vấn đề nâng cao trong Data Fitting	12
1.4.1 Đa cộng tuyến và hệ số VIF	12
1.4.2 Phân tích phần dư (Residual Analysis)	12
1.4.3 Cross Validation và lựa chọn mô hình	16
1.5 Mô tả thuật toán và cài đặt	17
1.5.1 Phương pháp OLS (Bình phương tối thiểu)	17
1.5.2 Đánh giá mô hình và kiểm định giả thuyết	19
1.5.3 Đa cộng tuyến, hồi quy Ridge và Lasso	20
1.5.4 Phân tích phần dư và lựa chọn mô hình	27
1.5.5 Kết quả Thực nghiệm so sánh và Biện luận chọn Mô hình	33
1.5.6 Minh hoạ Monte Carlo	34
2 PHẦN 2: ỨNG DỤNG THỰC TẾ TRÊN DỮ LIỆU	40
2.1 Tiêu Chí Chọn Bộ Dữ Liệu	40
2.2 Tiền Xử Lý Dữ Liệu	41
2.2.1 Khảo Sát Dữ Liệu (Exploratory Data Analysis — EDA)	41
2.2.2 Xử lý dữ liệu khuyết thiếu (Missing Values)	44
2.2.3 Trích xuất đặc trưng và tiền xử lý (Feature Engineering & Preprocessing)	45



2.3	Xây dựng mô hình và Kết quả thực nghiệm	46
2.3.1	Quy trình xây dựng mô hình	46
2.3.2	Các Mô Hình Cần Thử Nghiệm	48
2.3.3	Tiêu chí so sánh mô hình	56
2.4	Phân tích và Chẩn đoán Phần dư	58
2.4.1	Nhận xét chi tiết	58
3	PHẦN 3: KẾT LUẬN VÀ TÀI LIỆU THAM KHẢO	60
3.1	Phân tích sâu dữ liệu và đánh giá mô hình	60
3.2	Tài liệu tham khảo	60



Giới thiệu

Cấu trúc của báo cáo

Báo cáo này được chia thành các phần chính sau:

- **Phần 1: Lý Thuyết Data Fitting và Minh Họa**
- **Phần 2: Ứng Dụng Data Fitting vào Dữ Liệu Thực Tế**
- **Phần 3: Kết luận và Tài Liệu Tham Khảo .**

Kho báo cáo Github: https://github.com/KhoaTapCode2006/do_an_tudtk_02

Danh sách thành viên và Phân công công việc

STT	MSSV	Họ và Tên	Nhiệm vụ phân công	Mức độ hoàn thành
1	24120048	Lê Hoàng Hiếu	Cài đặt hàm tính VIF để xác định đa cộng tuyến, cài đặt hàm điều chỉnh Ridge, viết báo cáo cơ sở lý thuyết và cho hàm, tạo dữ liệu giả lập cho hàm để test	100%
2	24120131	Nguyễn Anh Sang	Cài đặt hàm mất mát tính nghiệm OLS, cài đặt hàm tính ma trận chiếu và Hat matrix, viết báo cáo cơ sở lý thuyết và cho hàm, tạo dữ liệu giả lập cho hàm để test	100%
3	24120075	Trần Nguyễn Anh Khoa	Cài đặt mô phỏng Monte Carlo, viết báo cáo cơ sở lý thuyết cho Gauss Markov, kiểm tra và sửa lỗi cho các hàm khác	100%
4	24120061	Lê Nguyễn Gia Huy	Cài đặt hàm tính hệ số xác định R^2 và các kiểm định giả thuyết, viết báo cáo cơ sở lý thuyết cho hàm, tạo dữ liệu giả lập cho hàm để test	100%
5	24120097	Nguyễn Hoàng Nam	Cài đặt hàm phân tích phần dư và thuật toán K-Fold Cross Validation, vẽ 4 biểu đồ để kiểm tra sai số mô hình, viết báo cáo cơ sở lý thuyết và cho hàm, tạo dữ liệu giả lập cho hàm để test	100%

Bảng 1: BẢNG PHÂN CÔNG CÔNG VIỆC NHÓM - PHẦN 1



STT	MSSV	Họ và Tên	Nhiệm vụ phân công	Mức độ hoàn thành
1	24120048	Lê Hoàng Hiếu	Viết cơ sở lý thuyết cho tiêu chí chọn bộ dữ liệu, khảo sát dữ liệu và thực hiện EDA, vẽ biểu đồ cho bộ dữ liệu	100%
2	24120131	Nguyễn Anh Sang	Lựa chọn và cài đặt phương pháp xử lý mất dữ liệu, cài đặt hàm thực hiện các bước tiền xử lý, chuẩn hoá cho dữ liệu, viết báo cáo cơ sở lý thuyết cho tiền xử lý và chuẩn hoá	100%
3	24120075	Trần Nguyễn Anh Khoa	Cài đặt 3 mô hình OLS cơ bản, từng biến và Ridge, phân tích sâu bộ dữ liệu, đưa ra đánh giá cho 3 mô hình và liên hệ thực tế, kiểm tra và sửa lỗi hàm	100%
4	24120061	Lê Nguyễn Gia Huy	Viết báo cáo cơ sở lý thuyết cho việc xây dựng mô hình, mô tả pipeline rõ ràng, chi tiết, vẽ biểu đồ hệ số hồi quy cho 3 mô hình	100%
5	24120097	Nguyễn Hoàng Nam	Viết báo cáo cơ sở lý thuyết cho tiêu chí so sánh mô hình, cài đặt hàm so sánh 3 mô hình sau khi đã được huấn luyện dựa trên bộ dữ liệu Yellow Taxi	100%

Bảng 2: BẢNG PHÂN CÔNG CÔNG VIỆC NHÓM - PHẦN 2

1 PHẦN 1: LÝ THUYẾT DATA FITTING VÀ MINH HỌA

1.1 Bài toán Data Fitting

1.1.1 Phát biểu bài toán tổng quát:

[Bài toán Data Fitting] Cho tập dữ liệu $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ với $x_i \in \mathbb{R}^p$ và $y_i \in \mathbb{R}$ [?]. Bài toán data fitting là tìm hàm $f: \mathbb{R}^p \rightarrow \mathbb{R}$ trong một lớp hàm cho trước sao cho f xấp xỉ tốt nhất ánh xạ từ x đến y_i theo một tiêu chí đã định.

Trong mô hình hồi quy tuyến tính, ta giả thiết mối quan hệ giữa các biến được thể hiện qua phương trình:

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip} + \epsilon_i = x_i^T \beta + \epsilon_i \quad (1)$$

Trong đó:

- $\beta = (\beta_0, \beta_1, \dots, \beta_p)^T \in \mathbb{R}^{p+1}$ là vector tham số cần ước lượng.
- ϵ_i là số hạng nhiễu ngẫu nhiên (sai số).

Để thuận tiện cho việc tính toán đại số tuyến tính, mô hình trên có thể được biểu diễn một cách gọn gàng dưới dạng ma trận:

$$y = X\beta + \epsilon \quad (2)$$

Trong đó:

- $y = (y_1, y_2, \dots, y_n)^T \in \mathbb{R}^n$ là vector biến mục tiêu.
- $X \in \mathbb{R}^{n \times (p+1)}$ là ma trận thiết kế (design matrix) với cột đầu tiên toàn số 1 đại diện cho hệ số chặn (intercept):

$$X = \begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1p} \\ 1 & x_{21} & x_{22} & \dots & x_{2p} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & x_{n2} & \dots & x_{np} \end{bmatrix}$$

- $\epsilon = (\epsilon_1, \epsilon_2, \dots, \epsilon_n)^T \in \mathbb{R}^n$ là vector nhiễu ngẫu nhiên.

1.1.2 Các Giả Thiết Gauss–Markov

Các Giả Thiết Gauss–Markov (GM1–GM5)

GM1. Tuyến tính: $y = X\beta + \epsilon$.

GM2. Không hoàn hảo đa cộng tuyến: $\text{rank}(X) = p + 1$, tức $X^T X$ khả nghịch.

GM3. Ngoại sinh (Exogeneity): $\mathbb{E}[\epsilon | X] = 0$.

GM4. Đồng phương sai (Homoscedasticity): $\text{Var}(\epsilon | X) = \sigma^2 I_n$, tức là $\text{Var}(\epsilon_i) = \sigma^2$ với mọi i , và $\text{Cov}(\epsilon_i, \epsilon_j) = 0$ với $i \neq j$.

GM5. Phần dư Chuẩn: $\epsilon | X \sim N(0, \sigma^2 I_n)$.

1.2 Phương pháp Ordinary Least Squares (OLS)

1.2.1 Hàm mất mát và Nghiệm OLS

Mô hình hồi quy tuyến tính:

Trong bài toán phân tích hồi quy tuyến tính đa biến, mô hình tổng quát được biểu diễn dưới dạng ma trận như sau:

$$y = X\beta + \epsilon \quad (3)$$

Trong đó:

- $\mathbf{y} \in \mathbb{R}^{n \times 1}$ là vector các giá trị quan sát (biến phụ thuộc).
- $\mathbf{X} \in \mathbb{R}^{n \times (p+1)}$ là ma trận thiết kế (design matrix) chứa các biến độc lập, thường có cột đầu tiên toàn số 1 để tương ứng với hệ số chặn (intercept).
- $\boldsymbol{\beta} \in \mathbb{R}^{(p+1) \times 1}$ là vector các hệ số hồi quy cần ước lượng.
- $\boldsymbol{\epsilon} \in \mathbb{R}^{n \times 1}$ là vector sai số ngẫu nhiên (nhiều).

Hàm mất mát (Loss Function):

Phương pháp Bình phương tối thiểu (Ordinary Least Squares - OLS) tìm kiếm một ước lượng $\hat{\boldsymbol{\beta}}$ sao cho tổng bình phương các phần dư (Residual Sum of Squares - RSS) là nhỏ nhất. Phần dư được định nghĩa là sự chênh lệch giữa giá trị thực tế và giá trị dự đoán: $\mathbf{e} = \mathbf{y} - \mathbf{X}\boldsymbol{\beta}$.

Hàm mất mát $S(\boldsymbol{\beta})$ được viết dưới dạng:

$$S(\boldsymbol{\beta}) = \sum_{i=1}^n \epsilon_i^2 = \boldsymbol{\epsilon}^T \boldsymbol{\epsilon} = (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^T (\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) \quad (4)$$

Khai triển biểu thức trên, ta có:

$$S(\boldsymbol{\beta}) = \mathbf{y}^T \mathbf{y} - \mathbf{y}^T \mathbf{X}\boldsymbol{\beta} - \boldsymbol{\beta}^T \mathbf{X}^T \mathbf{y} + \boldsymbol{\beta}^T \mathbf{X}^T \mathbf{X}\boldsymbol{\beta} \quad (5)$$

Vì $\boldsymbol{\beta}^T \mathbf{X}^T \mathbf{y}$ là một đại lượng vô hướng (scalar), nên nó bằng chuyển vị của chính nó: $(\boldsymbol{\beta}^T \mathbf{X}^T \mathbf{y})^T = \mathbf{y}^T \mathbf{X}\boldsymbol{\beta}$. Do đó:

$$S(\boldsymbol{\beta}) = \mathbf{y}^T \mathbf{y} - 2\boldsymbol{\beta}^T \mathbf{X}^T \mathbf{y} + \boldsymbol{\beta}^T \mathbf{X}^T \mathbf{X}\boldsymbol{\beta} \quad (6)$$

Nghiệm OLS:

Để tìm cực tiểu của hàm mất mát, ta lấy đạo hàm riêng của $S(\boldsymbol{\beta})$ theo vector $\boldsymbol{\beta}$ và cho bằng vector $\mathbf{0}$:

$$\frac{\partial S(\boldsymbol{\beta})}{\partial \boldsymbol{\beta}} = -2\mathbf{X}^T \mathbf{y} + 2\mathbf{X}^T \mathbf{X}\boldsymbol{\beta} = \mathbf{0} \quad (7)$$

Từ đây, ta thu được hệ phương trình chuẩn (Normal Equations):

$$\mathbf{X}^T \mathbf{X}\hat{\boldsymbol{\beta}} = \mathbf{X}^T \mathbf{y} \quad (8)$$

Giả sử ma trận $\mathbf{X}$ có hạng cột tối đa (full column rank), tức là $\mathbf{X}^T \mathbf{X}$ là ma trận khả nghịch. Khi đó, nghiệm duy nhất của bài toán OLS là:

$$\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \quad (9)$$

1.2.2 Ma Trận Chiếu và Hat Matrix

Định nghĩa Ma trận hình mũ (Hat Matrix):

Sau khi tìm được ước lượng $\hat{\beta}$, vector các giá trị dự đoán $\hat{\mathbf{y}}$ được tính bằng:

$$\hat{\mathbf{y}} = \mathbf{X}\hat{\beta} = \mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \quad (10)$$

Ta định nghĩa ma trận $\mathbf{H}$ như sau:

$$\mathbf{H} = \mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \quad (11)$$

Khi đó, $\hat{\mathbf{y}} = \mathbf{H}\mathbf{y}$. Ma trận $\mathbf{H}$ được gọi là *Hat Matrix* (Ma trận hình mũ) vì nó biến đổi ("đội mũ") vector quan sát thực tế $\mathbf{y}$ thành vector dự đoán $\hat{\mathbf{y}}$.

Ý nghĩa hình học (Ma trận chiếu):

Về mặt hình học, không gian sinh bởi các cột của $\mathbf{X}$ (column space của $\mathbf{X}$) chứa tất cả các tổ hợp tuyến tính có thể có của các đặc trưng. Vector $\mathbf{y}$ thực tế thường không nằm trong không gian này do có sai số ngẫu nhiên ϵ .

Ma trận $\mathbf{H}$ đóng vai trò là một **ma trận chiếu trực giao** (Orthogonal Projection Matrix). Nó chiếu vector $\mathbf{y}$ trong không gian $\mathbb{R}^n$ xuống không gian cột của $\mathbf{X}$ để tạo ra vector dự đoán $\hat{\mathbf{y}}$ sao cho khoảng cách (đại diện bởi phần dư $\mathbf{e}$) là ngắn nhất.

Các tính chất toán học quan trọng của Ma trận $\mathbf{H}$:

Ma trận chiếu $\mathbf{H}$ thỏa mãn hai tính chất cơ sở sau:

1. **Tính đối xứng (Symmetry):** $\mathbf{H}^T = \mathbf{H}$.

$$\mathbf{H}^T = (\mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T)^T = \mathbf{X} ((\mathbf{X}^T \mathbf{X})^{-1})^T \mathbf{X}^T = \mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T = \mathbf{H} \quad (12)$$

2. **Tính lũy đẳng (Idempotence):** $\mathbf{H}^2 = \mathbf{H}$. Về mặt hình học, việc chiếu lại một vector đã nằm sẵn trong không gian cột của $\mathbf{X}$ (do đã được chiếu 1 lần) sẽ không làm thay đổi

vector đó.

$$\mathbf{H}^2 = \mathbf{H}\mathbf{H} = (\mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T)(\mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T) \quad (13)$$

$$= \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}(\mathbf{X}^T\mathbf{X})(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T = \mathbf{X}\mathbf{I}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T = \mathbf{H} \quad (14)$$

1.2.3 Định Lý Gauss–Markov

Phát biểu định lý:

Dựa vào các giả thiết Gauss Markov từ GM1 đến GM4 (Tuyến tính, Không đa cộng tuyến, Ngoại sinh và Đồng phương sai), định lý Gauss-Markov phát biểu rằng trong một mô hình hồi quy tuyến tính cổ điển, các ước lượng bình phương nhỏ nhất thông thường (OLS) là ước lượng tuyến tính không chệch tốt nhất (BLUE - Best Linear Unbiased Estimator). Điều này có nghĩa là OLS có phương sai nhỏ nhất trong tất cả các ước lượng tuyến tính không chệch.

Điều này bao gồm 2 tính chất cốt lõi sau:

- **Tính không chệch (Unbiasedness):** Kỳ vọng của ước lượng bằng giá trị thực của tham số: $\mathbb{E}[\hat{\beta}_{OLS}] = \beta$.
- **Tính hiệu quả (Efficiency):** Trong lớp các ước lượng tuyến tính không chệch, ước lượng OLS có phương sai nhỏ nhất.

Giải thích 2 tính chất cốt lõi:

Tính không chệch (Unbiasedness) Một ước lượng được gọi là không chệch nếu giá trị kỳ vọng của nó bằng chính giá trị thực của tham số cần ước lượng.

$$\mathbb{E}[\hat{\beta}_{OLS}] = \beta \quad (15)$$

Ý nghĩa vật lý (Gauss Markov): Nếu chúng ta có thể lặp lại việc lấy mẫu vô số lần và tính toán hệ số $\hat{\beta}$ cho mỗi mẫu đó, thì giá trị trung bình của tất cả các hệ số này sẽ hội tụ về đúng giá trị thực β . Điều này đảm bảo rằng mô hình OLS không mắc phải *sai số hệ thống* (systematic error). Trong thực tế, mặc dù một lần chạy OLS cụ thể có thể cho kết quả hơi lệch, nhưng về mặt phương pháp, nó nhắm đúng vào "hồng tâm" của mục tiêu.

Tính hiệu quả (Efficiency) Tính hiệu quả liên quan đến độ phân tán (phương sai) của ước lượng. Trong lớp các ước lượng tuyến tính không chệch, OLS là ước lượng có phương sai nhỏ nhất.

$$\text{Var}(\hat{\beta}_{OLS}) \leq \text{Var}(\tilde{\beta}) \quad (16)$$

với mọi $\tilde{\beta}$ là ước lượng tuyến tính không chệch khác.

Ý nghĩa vật lý (Tính hiệu quả): Tính hiệu quả đại diện cho *độ chính xác* (precision). Một ước lượng hiệu quả hơn sẽ có các giá trị tập trung dày đặc quanh giá trị thực hơn là bị phân tán rộng.

- Nếu một phương pháp không chệch nhưng phương sai lớn, kết quả giữa các lần lấy mẫu sẽ rất bấp bênh (unreliable).
- OLS vừa không chệch, vừa có phương sai nhỏ nhất, nghĩa là nó cung cấp kết quả ổn định và đáng tin cậy nhất từ dữ liệu mẫu hiện có.

Ma trận hiệp phương sai của hệ số Phương sai của ước lượng $\hat{\beta}_{OLS}$ cho biết độ ổn định của các hệ số khi dữ liệu thay đổi. Công thức ma trận hiệp phương sai được xác định bởi:

$$Var(\hat{\beta}_{OLS}|X) = \sigma^2(X^T X)^{-1} \quad (17)$$

Trong đó σ^2 là phương sai của nhiễu ngẫu nhiên ϵ .

1.2.4 Ước lượng phương sai nhiễu

Trong thực tế, phương sai của nhiễu σ^2 thường không được biết trước và cần phải được ước lượng từ dữ liệu mẫu. Ước lượng không chệch của σ^2 , ký hiệu là $\hat{\sigma}^2$, được tính dựa trên tổng bình phương phần dư (RSS):

$$\hat{\sigma}^2 = \frac{RSS}{n - p - 1} = \frac{\|y - X\hat{\beta}\|^2}{n - p - 1} \quad (18)$$

Trong đó:

- n : Số lượng quan sát (mẫu dữ liệu).
- p : Số lượng biến độc lập (không tính hệ số chặn - intercept).
- $n - p - 1$: Số bậc tự do (Degrees of Freedom). Việc chia cho giá trị này thay vì n đảm bảo ước lượng là không chệch.

Bảng 3: Phân biệt thông số nhiều thực tế và ước lượng

Đặc điểm	Phương sai nhiều thực (σ^2)	Phương sai nhiều ước lượng ($\hat{\sigma}^2$)
Bản chất	Tham số ẩn của quần thể	Thống kê tính từ dữ liệu mẫu
Giá trị	Không đổi (theo GM4)	Thay đổi tùy theo mẫu dữ liệu
Công thức	$\text{Var}(\varepsilon_i)$	$\text{RSS}/(n - p - 1)$

1.3 Suy diễn thống kê và Đánh giá mô hình

1.3.1 Hệ số xác định R^2 và R^2 hiệu chỉnh

Hệ số xác định R^2 đo lường tỷ lệ biến thiên của biến phụ thuộc (y) được giải thích bởi các biến độc lập (X) trong mô hình hồi quy. Đại lượng này được tính dựa trên Tổng bình phương phần dư (RSS) và Tổng bình phương toàn phần (TSS):

$$R^2 = 1 - \frac{RSS}{TSS} = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (19)$$

Giá trị R^2 luôn nằm trong khoảng $[0, 1]$. Tuy nhiên, R^2 có một nhược điểm lớn: nó sẽ không bao giờ giảm và thường có xu hướng tăng khi ta thêm các biến độc lập mới vào mô hình, ngay cả khi các biến đó không thực sự có ý nghĩa. Để khắc phục vấn đề này, ta sử dụng hệ số R^2 hiệu chỉnh ($\bar{R}^2$ hay Adjusted R^2). Công thức này áp dụng một "hình phạt" đối với việc thêm vào các biến không cần thiết bằng cách đưa bậc tự do vào tính toán:

$$\bar{R}^2 = 1 - \left(\frac{n-1}{n-p-1} \right) (1 - R^2) \quad (20)$$

trong đó, n là tổng số quan sát và p là số lượng biến độc lập trong mô hình.

1.3.2 Kiểm định giả thuyết (t-test, F-test)

Kiểm định Student (t-test) cho từng hệ số hồi quy:

Kiểm định t được sử dụng để đánh giá ý nghĩa thống kê của từng hệ số hồi quy riêng lẻ β_j . Cặp giả thuyết thống kê được đặt ra là:

- $H_0 : \beta_j = 0$ (Biến độc lập thứ j không có tác động đến biến phụ thuộc)
- $H_1 : \beta_j \neq 0$ (Biến độc lập thứ j thực sự có tác động)

Giá trị thống kê t (t-statistic) được xác định bằng tỷ số giữa giá trị ước lượng của hệ số và

sai số chuẩn (Standard Error - SE) của nó:

$$t_j = \frac{\hat{\beta}_j}{SE(\hat{\beta}_j)} \quad (21)$$

Dưới giả thuyết H_0 , thống kê t_j tuân theo phân phối Student với bậc tự do $df = n - p - 1$.

Cách tính p-value: Vì giả thuyết đối (H_1) là phép thử khác ($\neq$), đây là một **kiểm định hai phía (two-tailed test)**. Giá trị p -value chính là xác suất để một biến ngẫu nhiên tuân theo phân phối Student có giá trị tuyệt đối lớn hơn hoặc bằng giá trị $|t_j|$ tính được từ mẫu. Bằng cách sử dụng Hàm phân phối tích lũy (CDF) của phân phối Student, p -value được tính như sau:

$$p\text{-value}_j = 2 \times P(T > |t_j|) = 2 \times (1 - \text{CDF}_{t_{n-p-1}}(|t_j|)) \quad (22)$$

Nếu $p\text{-value}_j < \alpha$ (với α thường chọn là 0.05), ta có đủ cơ sở để bác bỏ giả thuyết H_0 , và kết luận rằng hệ số β_j có ý nghĩa thống kê.

Kiểm định Fisher (F-test) cho sự phù hợp tổng thể:

Thay vì kiểm tra từng biến lẻ tẻ, kiểm định F đánh giá xem liệu *tất cả* các biến độc lập trong mô hình có cùng nhau tạo ra một mô hình có ý nghĩa hay không. Cặp giả thuyết được đặt ra là:

- $H_0 : \beta_1 = \beta_2 = \dots = \beta_p = 0$ (Mô hình hoàn toàn vô nghĩa)
- $H_1 : \text{Tồn tại ít nhất một } \beta_j \neq 0 \text{ (} j = 1, \dots, p \text{)}$ (Mô hình có ý nghĩa)

Giá trị thống kê F được tính dựa trên tỷ số giữa Trung bình bình phương được giải thích (Model Mean Square) và Trung bình bình phương phần dư (Mean Square Error):

$$F = \frac{(TSS - RSS)/p}{RSS/(n - p - 1)} \quad (23)$$

Dưới giả thuyết H_0 , thống kê F tuân theo phân phối Fisher-Snedecor với hai bậc tự do là $df_1 = p$ (bậc tự do của tử số) và $df_2 = n - p - 1$ (bậc tự do của mẫu số).

Cách tính p-value: Do bản chất của phân phối F và cách thiết lập thống kê (các cấu phần đều là bình phương dương), kiểm định F luôn là một **kiểm định một phía bên phải (right-tailed test)**. Giá trị F càng lớn thì bằng chứng chống lại H_0 càng mạnh. Do đó, p -value là diện tích phần đuôi bên phải của đường cong phân phối F tính từ giá trị thống kê F vừa tìm được:

$$p\text{-value}_F = P(F_{p, n-p-1} > F) = 1 - \text{CDF}_{F_{p, n-p-1}}(F) \quad (24)$$

Nếu $p\text{-value}_F < \alpha$, ta bác bỏ H_0 , đồng nghĩa với việc mô hình hồi quy tuyến tính được xây dựng có khả năng giải thích sự biến thiên của dữ liệu tốt hơn việc chỉ dùng giá trị trung bình.

1.4 Các vấn đề nâng cao trong Data Fitting

1.4.1 Đa cộng tuyến và hệ số VIF

Đa cộng tuyến xảy ra khi một biến độc lập có thể được giải thích gần đúng bởi các biến độc lập còn lại. Khi đó ma trận $\mathbf{X}^\top \mathbf{X}$ trở nên gần suy biến, làm cho phương sai của các ước lượng OLS tăng mạnh. Hệ quả là hệ số hồi quy có thể dao động lớn khi dữ liệu thay đổi nhỏ, dấu của hệ số có thể khó diễn giải và kiểm định t cho từng hệ số trở nên kém ổn định.

Với biến độc lập X_j , ta hồi quy phụ X_j theo toàn bộ các biến độc lập còn lại và thu được hệ số xác định R_j^2 . Hệ số phóng đại phương sai (*Variance Inflation Factor*) của biến X_j được định nghĩa bởi:

$$\text{VIF}_j = \frac{1}{1 - R_j^2}. \quad (25)$$

Nếu R_j^2 càng gần 1, biến X_j càng được giải thích tốt bởi các biến còn lại, nên VIF_j càng lớn. Trong thực hành, một số quy ước hay dùng là $\text{VIF}_j > 5$ cho thấy dấu hiệu đa cộng tuyến đáng chú ý, và $\text{VIF}_j > 10$ cho thấy đa cộng tuyến nghiêm trọng. Khi $R_j^2 = 1$, biến X_j là tổ hợp tuyến tính hoàn hảo của các biến khác và VIF tiến tới vô hạn.

VIF liên hệ trực tiếp với độ bất ổn của ước lượng OLS. Nếu giả sử các biến đã được chuẩn hóa, phương sai của hệ số $\hat{\beta}_j$ có dạng tỉ lệ với VIF_j :

$$\text{Var}(\hat{\beta}_j) \propto \frac{\sigma^2}{\sum_{i=1}^n (x_{ij} - \bar{x}_j)^2} \text{VIF}_j. \quad (26)$$

Vì vậy, VIF không chỉ phát hiện quan hệ tuyến tính giữa các biến độc lập, mà còn giải thích vì sao đa cộng tuyến làm tăng sai số chuẩn của hệ số hồi quy.

1.4.2 Phân tích phần dư (Residual Analysis)

Phần dư là sai khác giữa giá trị thực tế và giá trị dự đoán của mô hình:

$$e_i = y_i - \hat{y}_i, \quad \mathbf{e} = \mathbf{y} - \hat{\mathbf{y}}. \quad (27)$$

Phân tích phần dư dùng để kiểm tra xem mô hình hồi quy tuyến tính có vi phạm các giả thiết quan trọng như tuyến tính, phương sai không đổi, độc lập sai số và phân phối chuẩn xấp xỉ của

sai số hay không.

Trong mô hình hồi quy tuyến tính đa biến $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$ với giả thiết Gauss–Markov ($\mathbb{E}[\boldsymbol{\varepsilon} | \mathbf{X}] = \mathbf{0}$ và $\text{Var}(\boldsymbol{\varepsilon} | \mathbf{X}) = \sigma^2 \mathbf{I}_n$), các sai số ngẫu nhiên $\boldsymbol{\varepsilon}$ là các đại lượng lý thuyết không thể quan sát trực tiếp. Do đó, ta phải đánh giá chúng gián tiếp thông qua vector phần dư quan sát được $\hat{\mathbf{e}}$:

$$\hat{\mathbf{e}} = \mathbf{y} - \hat{\mathbf{y}} = \mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}. \quad (28)$$

Sử dụng ma trận Hat Matrix $\mathbf{H} = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$, ta có $\hat{\mathbf{y}} = \mathbf{H}\mathbf{y}$. Từ đó, vector phần dư có thể được biểu diễn như một phép chiếu trực giao trên không gian trực giao của cột ma trận thiết kế:

$$\hat{\mathbf{e}} = \mathbf{y} - \mathbf{H}\mathbf{y} = (\mathbf{I}_n - \mathbf{H})\mathbf{y}. \quad (29)$$

Thế phương trình thực tế $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$ vào, ta có:

$$\hat{\mathbf{e}} = (\mathbf{I}_n - \mathbf{H})(\mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}) = (\mathbf{I}_n - \mathbf{H})\mathbf{X}\boldsymbol{\beta} + (\mathbf{I}_n - \mathbf{H})\boldsymbol{\varepsilon}. \quad (30)$$

Vì $\mathbf{H}\mathbf{X} = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{X} = \mathbf{X}$, suy ra $(\mathbf{I}_n - \mathbf{H})\mathbf{X} = \mathbf{0}$. Do đó, phương trình trên rút gọn thành mối quan hệ tuyến tính trực tiếp giữa phần dư và sai số thực tế:

$$\hat{\mathbf{e}} = (\mathbf{I}_n - \mathbf{H})\boldsymbol{\varepsilon}. \quad (31)$$

Từ hệ thức này, dưới các giả thiết Gauss–Markov, ta thu được các tính chất thống kê quan trọng của phần dư:

- **Kỳ vọng của phần dư:** Do $\mathbb{E}[\boldsymbol{\varepsilon} | \mathbf{X}] = \mathbf{0}$, ta có kỳ vọng không chệch:

$$\mathbb{E}[\hat{\mathbf{e}} | \mathbf{X}] = \mathbb{E}[(\mathbf{I}_n - \mathbf{H})\boldsymbol{\varepsilon} | \mathbf{X}] = (\mathbf{I}_n - \mathbf{H})\mathbb{E}[\boldsymbol{\varepsilon} | \mathbf{X}] = \mathbf{0}. \quad (32)$$

- **Ma trận hiệp phương sai của phần dư:**

$$\text{Var}(\hat{\mathbf{e}} | \mathbf{X}) = \text{Var}((\mathbf{I}_n - \mathbf{H})\boldsymbol{\varepsilon} | \mathbf{X}) = (\mathbf{I}_n - \mathbf{H}) \text{Var}(\boldsymbol{\varepsilon} | \mathbf{X}) (\mathbf{I}_n - \mathbf{H})^\top. \quad (33)$$

Vì $\text{Var}(\boldsymbol{\varepsilon} | \mathbf{X}) = \sigma^2 \mathbf{I}_n$ và ma trận $(\mathbf{I}_n - \mathbf{H})$ là ma trận đối xứng, lũy đẳng (tức là $(\mathbf{I}_n - \mathbf{H})^\top = \mathbf{I}_n - \mathbf{H}$ và $(\mathbf{I}_n - \mathbf{H})^2 = \mathbf{I}_n - \mathbf{H}$), ta có:

$$\text{Var}(\hat{\mathbf{e}} | \mathbf{X}) = \sigma^2 (\mathbf{I}_n - \mathbf{H})(\mathbf{I}_n - \mathbf{H}) = \sigma^2 (\mathbf{I}_n - \mathbf{H}). \quad (34)$$

Từ ma trận hiệp phương sai trên, phương sai của phần dư thứ i và hiệp phương sai giữa hai phần dư thứ i, j ($i \neq j$) được xác định bởi:

$$\text{Var}(e_i) = \sigma^2(1 - h_{ii}), \quad \text{Cov}(e_i, e_j) = -\sigma^2 h_{ij}, \quad (35)$$

trong đó h_{ii} và h_{ij} tương ứng là các phần tử đường chéo và ngoài đường chéo của ma trận Hat Matrix $\mathbf{H}$. Điều này dẫn đến hai nhận xét cốt lõi:

1. **Phương sai không đồng nhất:** Các phần dư e_i có phương sai khác nhau phụ thuộc vào giá trị leverage h_{ii} của từng quan sát.
2. **Các phần dư bị phụ thuộc tuyến tính:** Hiệp phương sai giữa các phần dư khác nhau nói chung là khác không ($\text{Cov}(e_i, e_j) \neq 0$), nghĩa là các phần dư có mối tương quan ràng buộc với nhau.

Để đưa các phần dư về cùng một thang đo có phương sai đồng nhất bằng 1 nhằm phục vụ chẩn đoán trực quan chính xác, ta định nghĩa các đại lượng phần dư chuẩn hóa và leverage:

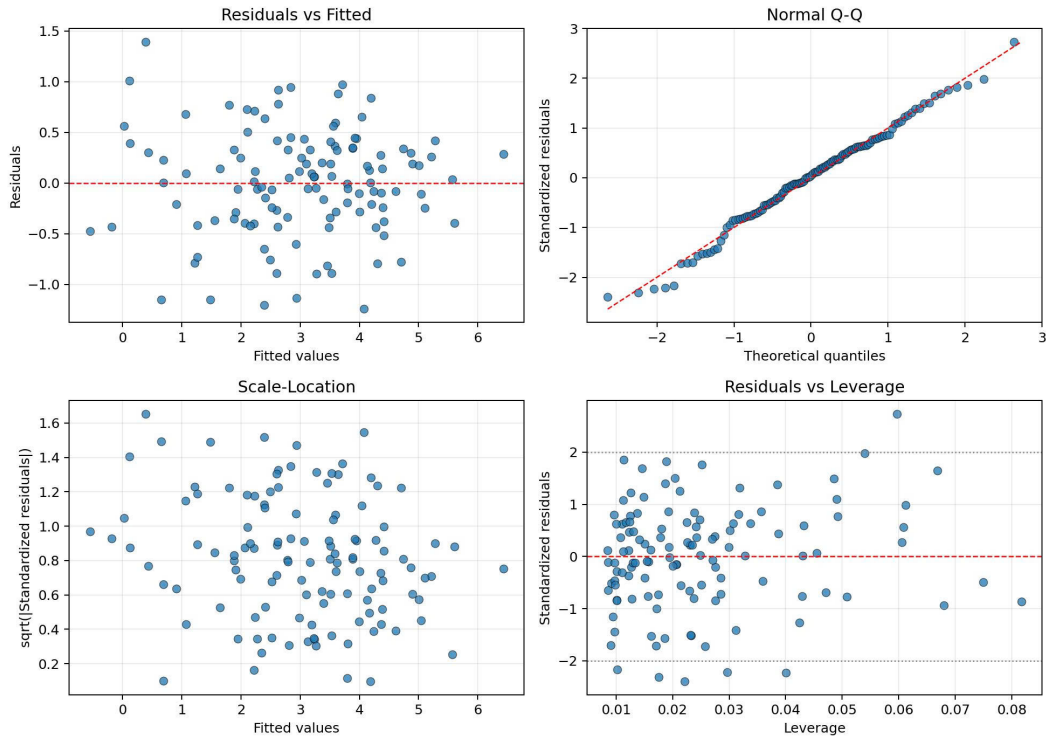
- **Phần dư thô:** $e_i = y_i - \hat{y}_i$.
- **Phần dư chuẩn hóa:**

$$r_i = \frac{e_i}{\sigma \sqrt{1 - h_{ii}}},$$

trong đó h_{ii} là phần tử đường chéo của ma trận hat matrix.

- **Leverage:** h_{ii} đo mức độ quan sát thứ i nằm xa vùng trung tâm của không gian biến độc lập.

Four Residual Diagnostic Plots



Hình 1: Biểu đồ Residuals vs Fitted, Normal Q-Q, Scale-Location và Residuals vs Leverage.

Ý nghĩa của các biểu đồ:

- 1. Residuals vs Fitted Plot:** Biểu diễn cặp tọa độ $(\hat{y}_i, e_i)$. Giúp kiểm tra giả thiết về mối quan hệ tuyến tính. Nếu giả thiết đúng, các điểm phần dư phải phân tán ngẫu nhiên, không tạo thành quy luật hình học (parabol, phễu) và đường xu hướng làm mịn bằng thuật toán Moving Average phải phẳng tiệm cận trục hoành $e = 0$.
- 2. Normal Q-Q Plot:** Trực quan hóa mối tương quan giữa phân vị thực tế của phần dư chuẩn hóa và phân vị lý thuyết từ phân phối chuẩn tắc $\mathcal{N}(0, 1)$. Nếu thành phần nhiễu đạt chuẩn tắc, các điểm sẽ bám sát đường thẳng tham chiếu 45 độ ($y = x$).
- 3. Scale-Location Plot:** Biểu diễn cặp tọa độ $(\hat{y}_i, \sqrt{|r_i|})$. Mục tiêu là kiểm chứng giả thiết phương sai sai số đồng nhất (Homoscedasticity). Đường xu hướng phẳng nằm ngang biểu thị phương sai bất biến; ngược lại, nếu tạo thành hình phễu mở rộng, mô hình đã vi phạm giả thiết và xảy ra hiện tượng phương sai thay đổi.

4. **Residuals vs Leverage Plot:** Biểu diễn cặp tọa độ (h_{ii}, r_i) . Đây là công cụ mạnh mẽ để phân tách giữa điểm dị biệt hình học (Outliers) và điểm có sức ảnh hưởng lớn (Influential Points). Bằng cách lồng ghép các đường biên khoảng cách Cook (Cook's Distance D_i) tại các mức ý nghĩa 0.5 và 1.0 dựa trên hệ thức:

$$D_i = \frac{r_i^2}{p+1} \left(\frac{h_{ii}}{1-h_{ii}} \right) \quad (36)$$

Biểu đồ cho phép xác định các quan sát vượt biên làm bóp méo nghiệm OLS thực tế.

1.4.3 Cross Validation và lựa chọn mô hình

Để ngăn chặn hiện tượng quá khớp (Overfitting) – trường hợp mô hình ghi nhớ cả nhiễu của tập dữ liệu huấn luyện thay vì học quy luật tổng quát, phương pháp k -Fold Cross-Validation được thực hiện nhằm ước lượng khách quan hiệu năng ngoại mẫu (out-of-sample).

Thuật toán phân ngẫu nhiên không gian mẫu kích thước n thành k tập con độc lập $\{F_1, F_2, \dots, F_k\}$ có kích thước xấp xỉ bằng nhau. Tại mỗi vòng lặp thứ $i \in \{1, \dots, k\}$, mô hình OLS được tối ưu hóa trên tập nền $\bigcup_{j \neq i} F_j$ để sinh ra hệ số $\hat{\beta}^{(-i)}$. Chỉ số hiệu năng R^2 của riêng fold đó được đánh giá độc lập trên tập kiểm chứng F_i theo hệ thức:

$$R_{(i)}^2 = 1 - \frac{\sum_{m \in F_i} (y_m - \mathbf{x}_m^T \hat{\beta}^{(-i)})^2}{\sum_{m \in F_i} (y_m - \bar{y}_{F_i})^2} \quad (37)$$

Hiệu năng tổng thể của mô hình được biểu diễn qua giá trị trung bình $\bar{R}^2$ và độ lệch chuẩn $\text{STD}(R^2)$:

$$\bar{R}^2 = \frac{1}{k} \sum_{i=1}^k R_{(i)}^2, \quad \text{STD}(R^2) = \sqrt{\frac{1}{k} \sum_{i=1}^k (R_{(i)}^2 - \bar{R}^2)^2} \quad (38)$$

Nhằm nâng cao tính khoa học cho việc lựa chọn mô hình, bên cạnh độ đo thực nghiệm Cross-Validation, nhóm áp dụng thêm hai tiêu chí lý thuyết thông tin dựa trên hàm hợp lý cực đại kết hợp hình phạt bậc tự do là AIC và BIC:

$$\text{AIC} = n \ln \left(\frac{\text{RSS}}{n} \right) + 2(p+1) \quad (39)$$

$$\text{BIC} = n \ln \left(\frac{\text{RSS}}{n} \right) + (p+1) \ln(n) \quad (40)$$

Một mô hình được coi là tối ưu cân bằng giữa độ khớp (goodness of fit) và độ phức tạp nếu tối

đa hóa được $\bar{R}^2$ đồng thời tối thiểu hóa bộ đôi chỉ số AIC và BIC.

1.5 Mô tả thuật toán và cài đặt

1.5.1 Phương pháp OLS (Bình phương tối thiểu)

a. Cài đặt hàm `ols_fit(X, y)`:

Mô tả thuật toán:

1. Chuyển đổi dữ liệu đầu vào $\mathbf{X}$ và $\mathbf{y}$ sang dạng list, đồng thời định dạng lại $\mathbf{y}$ thành vector cột kích thước $n \times 1$.
2. Xác định số lượng quan sát n và số lượng biến độc lập p (bỏ qua cột hệ số chặn - intercept).
3. Tính ma trận chuyển vị $\mathbf{X}^T$ và ma trận vuông $\mathbf{X}^T \mathbf{X}$.
4. Tìm ma trận nghịch đảo $(\mathbf{X}^T \mathbf{X})^{-1}$.
5. Tính vector hệ số hồi quy theo công thức $\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$.
6. Tìm vector giá trị dự đoán $\hat{\mathbf{y}} = \mathbf{X} \hat{\beta}$ và tính phần dư $\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}}$.
7. Tính tổng bình phương phần dư (RSS) và ước lượng phương sai sai số $\hat{\sigma}^2 = \frac{RSS}{n-p-1}$.

```
1 def ols_fit(X, y):
2     X_list = X.tolist()
3     y_list = y.tolist()
4
5     # Dưa y về dạng vector cột
6     if len(np.array(y_list).shape) == 1:
7         y_list = [[val] for val in y_list]
8
9     n = len(X_list)
10    p = len(X_list[0]) - 1 # Bỏ qua cột intercept
11
12    XT = transpose(X_list)
13    XT_X = matmul(XT, X_list)
14    XT_X_inv = invert_matrix(XT_X)
15    XT_y = matmul(XT, y_list)
16
```

```
17 # Tính beta_hat
18 beta_hat = matmul(XT_X_inv, XT_y)
19
20 # Tính sigma2_hat
21 y_hat = matmul(X_list, beta_hat)
22 residuals = [[y_list[i][0] - y_hat[i][0]] for i in range(n)]
23 rss = sum(r[0]**2 for r in residuals)
24 sigma2_hat = rss / (n - p - 1)
25
26 return np.array(beta_hat), float(sigma2_hat)
```

b. Cài đặt hàm hat_matrix(X):

Mô tả thuật toán:

1. Tính ma trận chuyển vị $\mathbf{X}^T$ từ ma trận thiết kế $\mathbf{X}$.
2. Nhân ma trận $\mathbf{X}^T$ với $\mathbf{X}$ để thu được $\mathbf{X}^T \mathbf{X}$.
3. Tìm ma trận nghịch đảo $(\mathbf{X}^T \mathbf{X})^{-1}$.
4. Nhân lần lượt các ma trận từ trái sang phải: đầu tiên tính $\mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1}$, sau đó nhân kết quả với $\mathbf{X}^T$ để thu được ma trận chiếu $\mathbf{H}$.

```
1 def hat_matrix(X):
2     X_list = X.tolist()
3     XT = transpose(X_list)
4     XT_X = matmul(XT, X_list)
5
6     # Nghịch đảo ma trận X^T X
7     XT_X_inv = invert_matrix(XT_X)
8
9     # Tính X (X^T X)^-1 X^T
10    part1 = matmul(X_list, XT_X_inv)
11    H = matmul(part1, XT)
12
13    return np.array(H)
```

1.5.2 Đánh giá mô hình và kiểm định giả thuyết

Listing 1: 1 vài hàm hỗ trợ

```
1 calculate\_tss(y) # Tính tổng đo lech bình phương
2 calculate\_rss(y, y_hat) # Tính tổng bình phương phần dư
3 flatten(list\_of\_lists) # Chuyển 1 list đa chiều thành list 1 chiều
```

a. Hàm `model_metrics(y, y_hat, p)` Mô tả thuật toán:

- **Bước 1 (Tính tổng bình phương):** Xác định tổng bình phương toàn phần $TSS = \sum_{i=1}^n (y_i - \bar{y})^2$ và tổng bình phương phần dư $RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2$.
- **Bước 2 (Tính hệ số giải thích):** Dựa vào TSS và RSS , tính hệ số xác định $R^2 = 1 - \frac{RSS}{TSS}$ và hệ số hiệu chỉnh $\bar{R}^2 = 1 - \frac{n-1}{n-p-1}(1 - R^2)$.
- **Bước 3 (Kiểm định F):** Tính toán giá trị thống kê $F = \frac{(TSS-RSS)/p}{RSS/(n-p-1)}$. Trong trường hợp mô hình khớp hoàn hảo ($RSS \approx 0$), thuật toán tự động gán $F = \infty$ để ngăn chặn lỗi chia cho 0 (`ZeroDivisionError`).
- **Đầu ra:** Từ điển (Dictionary) chứa các chỉ số vô hướng đo lường mô hình.

Mã nguồn cài đặt:

Listing 2: Hàm `model_metrics(y, y_hat, p)`

```
1 def model_metrics(y, y_hat, p):
2     """
3     Danh gia do chinh xac cua mo hinh hoi quy OLS
4     """
5     y = flatten(y)
6     y_hat = flatten(y_hat)
7
8     n = len(y)
9
10    TSS = calculate_tss(y)
11    RSS = calculate_rss(y, y_hat)
12
13    R2 = 1 - (RSS / TSS)
14    Adj_R2 = 1 - ((n - 1) / (n - p - 1)) * (1 - R2)
```

```
15
16 if np.isclose(RSS, 0.0):
17     F_stat = np.inf
18 else:
19     F_stat = ((TSS - RSS) / p) / (RSS / (n - p - 1))
20
21 return {
22     "RSS": float(np.round(RSS, 4)),
23     "TSS": float(np.round(TSS, 4)),
24     "R2": float(np.round(R2, 4)),
25     "Adj_R2": float(np.round(Adj_R2, 4)),
26     "F_stat": float(np.round(F_stat, 4))
27 }
```

b. Cài đặt hàm `coef_inference(X, y, beta_hat, sigma2_hat)`

Mô tả thuật toán:

- **Đầu vào:** Ma trận đặc trưng X , vector mục tiêu y , vector hệ số đã ước lượng $\hat{\beta}$ và ước lượng phương sai nhiễu $\hat{\sigma}^2$.
- **Bước 1 (Tính ma trận hiệp phương sai):** Tính ma trận nghịch đảo $C = (X^T X)^{-1}$. Thuật toán ưu tiên sử dụng ma trận giả nghịch đảo (Moore-Penrose pseudoinverse) để đảm bảo tính ổn định trong trường hợp dữ liệu bị đa cộng tuyến.
- **Bước 2 (Sai số chuẩn và thống kê t):** Tính sai số chuẩn $SE(\hat{\beta}_j) = \sqrt{\hat{\sigma}^2 C_{jj}}$ dựa trên các phần tử nằm trên đường chéo chính của ma trận C . Sau đó tính trị thống kê $t_j = \frac{\hat{\beta}_j}{SE(\hat{\beta}_j)}$.
- **Bước 3 (Tính p-value và Khoảng tin cậy):** Tra cứu hàm phân phối Student thông qua hàm sinh tồn (Survival function - `sf`) để lấy giá trị p -value song diện. Tiếp theo, tính biên dưới và biên trên cho khoảng tin cậy 95% dựa trên giá trị tới hạn t_{crit} .
- **Đầu ra:** Cấu trúc bảng Pandas `DataFrame` tổng hợp kết quả kiểm định.

1.5.3 Đa cộng tuyến, hồi quy Ridge và Lasso

a. Hàm `vif(X)`

Mô tả thuật toán: Chạy vòng lặp từ $j = 1$ đến p (Với p là số lượng biến độc lập), mỗi vòng lặp thực hiện:

1. Tách X_j làm biến mục tiêu của hồi quy phụ
2. Dùng các cột $X_k, k \neq j$ làm biến giải thích và thêm intercept
3. Ước lượng mô hình phụ bằng OLS
4. Tính $R_j^2 = 1 - RSS_j/TSS_j$
5. Tính $VIF_j = 1/(1 - R_j^2)$

Kết thúc vòng lặp và trả về: Dictionary các giá trị VIF

Mã nguồn cài đặt.

Listing 3: 1 vài hàm hỗ trợ

```
1 _as_matrix(values, name) # Chuyển đổi dữ liệu đầu vào thành ma trận
2 _as_vector(values, name) # Chuyển đổi dữ liệu đầu vào thành vector
3 _transpose(matrix) # Ma trận chuyển vị
4 __matmul(left, right) # Nhân 2 ma trận
5 _matvec(matrix, vector) # Nhân ma trận với vector
6 _solve_linear_system(matrix, vector, tol=1e-12) # Hàm giải phương trình tuyến tính
7 _rss(actual, predicted) # Tính tổng bình phương phần dư
8 _tss(values) # Tính tổng bình phương toàn bộ
```

a. Cài đặt hàm vif(X)

Listing 4: Mã nguồn hàm vif()

```
1 def vif(X):
2
3     # Lấy tên của các biến độc lập
4     names = _feature_names(X, len(X.columns) if hasattr(X, "columns") else 0)
5     # Chuyển đổi dữ liệu đầu vào thành ma trận dùng kiểu list long list của Python
6     matrix = _as_matrix(X, "X")
7     n_rows = len(matrix)
8     n_cols = len(matrix[0])
9     if len(names) != n_cols:
10         names = _feature_names(X, n_cols)
11     if n_rows < 2:
12         raise ValueError("X must contain at least two observations.")
```

```
13 # Neu chi co 1 bien doc lap thi khong co da cong tuyen, VIF luon bang 1.0
14 if n_cols == 1:
15     return {names[0]: 1.0}
16
17 result = {}
18 # Vong lap duyet qua tung bien de coi bien do nhu la bien muc tieu (bien y gia)
19 for target_col in range(n_cols):
20     # Tach cot cua bien hien tai ra de lam bien muc tieu y
21     target = [row[target_col] for row in matrix]
22     # Lay chi so cua tat ca cac bien con lai de lam cac bien doc lap X
23     other_cols = [
24         index for index in range(n_cols) if index != target_col
25     ]
26     # Tao ma tran thiet ke gom cot toan so 1 (intercept) va cac bien con lai
27     design = [
28         [1.0] + [row[index] for index in other_cols]
29         for row in matrix
30     ]
31
32     # Tinh Tong binh phuong toan bo (TSS) cua bien muc tieu gia
33     total = _tss(target)
34     # Neu bien nay khong thay doi (phuong sai = 0), no gay ra da cong tuyen hoan hao
35     if total < 1e-12:
36         result[names[target_col]] = float("inf")
37         continue
38
39     try:
40         # Khop mo hinh OLS lay bien hien tai hoi quy theo cac bien con lai
41         beta = _ols_fit(design, target)
42         # Tinh gia tri du doan cho bien hien tai
43         predicted = _matvec(design, beta)
44         # Tinh he so xac dinh R-squared (R2) cua mo hinh phu nay
45         r_squared = 1.0 - _rss(target, predicted) / total
46     except ValueError:
47         # Neu ma tran suy bien khong giai duoc thi VIF la vo cung (inf)
48         result[names[target_col]] = float("inf")
```

```
49         continue
50
51         # Neu R-squared gan bang 1, nghĩa là biến này bị giải thích hoàn-hao bởi các
           biến khác
52         if r_squared >= 1.0 - 1e-10:
53             result[names[target_col]] = float("inf")
54         else:
55             # Áp dụng công thức tính VIF = 1 / (1 - R2)
56             r_squared = max(0.0, min(r_squared, 1.0 - 1e-10))
57             result[names[target_col]] = 1.0 / (1.0 - r_squared)
58
59     # Trả về tu diện chứa hệ số VIF của từng biến
60     return result
```

b. Cài đặt hàm `ridge_fit(X, y, lam)`

Mô tả thuật toán

1. Tính $\mathbf{X}^\top \mathbf{X}$ và $\mathbf{X}^\top \mathbf{y}$
2. Xác định ma trận phạt $\mathbf{P}$
3. Tạo hệ phương trình $(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{P})\boldsymbol{\beta} = \mathbf{X}^\top \mathbf{y}$
4. Giải hệ bằng khử Gauss-Jordan có chọn pivot
5. Trả về $\hat{\boldsymbol{\beta}}_{\text{ridge}}$

Mã nguồn cài đặt

Listing 5: Cài đặt hàm `ridge_fit()`

```
1 def ridge_fit(X, y, lam):
2     """Estimate Ridge regression coefficients."""
3
4     if lam < 0:
5         raise ValueError("lam must be non-negative.")
6
7     # Chuyển đổi ma trận đặc trưng X và vector mục tiêu y về dạng list phù hợp
8     matrix = _as_matrix(X, "X")
```



```
9 target = _as_vector(y, "y")
10 if len(matrix) != len(target):
11     raise ValueError("X and y must have the same number of rows.")
12
13 # Tính các thành phần cơ bản của phương trình chuẩn: X^T và X^T * X
14 xt = _transpose(matrix)
15 xtx = _matmul(xt, matrix)
16 xty = _matvec(xt, target)
17
18 # Kiểm tra xem cột đầu tiên có phải là cột Intercept (toán số 1) hay không
19 # Nếu có Intercept thì bắt đầu phạt (penalize) từ cột thứ hai (index = 1)
20 # Nếu không có Intercept thì phạt tất cả các cột (index = 0)
21 first_penalized = 1 if _has_intercept_column(matrix) else 0
22
23 # Cộng thêm giá trị lambda (lam) vào các phần tử trên đường chéo chính của ma trận
24 # Đây chính là bước thực hiện hình thức Regularization của Ridge: (X^T*X + lam*I)
25 for index in range(first_penalized, len(xtx)):
26     xtx[index][index] += float(lam)
27
28 try:
29     # Giải hệ phương trình tuyến tính bằng khu Gauss để tìm các hệ số beta Ridge
30     return _solve_linear_system(xtx, xty)
31 except ValueError as exc:
32     raise ValueError(
33         "The Ridge system is singular. Try lam > 0 or remove duplicate "
34         "columns from X."
35     ) from exc
```

c. Cài đặt hàm `lasso_fit(X, y, lam)` (Sử dụng Coordinate Descent)

Mô tả thuật toán: Hồi quy Lasso giải bài toán tối ưu hóa có dạng:

$$\min_{\beta} \left\{ \frac{1}{2n} \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \lambda \sum_{j \in \mathcal{P}} |\beta_j| \right\} \quad (41)$$

Các bước thực hiện:

1. Khởi tạo vector hệ số $\beta^{(0)} = \mathbf{0}$.
2. Xác định cột đầu tiên của $\mathbf{X}$ có phải cột intercept (toàn số 1) hay không. Thiết lập chỉ số phạt bắt đầu từ 1 nếu có intercept, ngược lại bắt đầu từ 0.
3. Tính độ tự tương quan của mỗi cột đặc trưng: $a_j = \frac{1}{n} \sum_{i=1}^n X_{ij}^2$ với mọi $j = 0, \dots, p$.
4. Khởi tạo vector phần dư $\mathbf{e}^{(0)} = \mathbf{y} - \mathbf{X}\beta^{(0)} = \mathbf{y}$.
5. Vòng lặp tối ưu hóa cập nhật hệ số cho đến khi hội tụ: Với mỗi đặc trưng $j = 0, \dots, p$:
 - (a) Tính đại lượng hiệp phương sai phần dư: $\rho_j = \frac{1}{n} \sum_{i=1}^n X_{ij}e_i$.
 - (b) Tính giá trị cập nhật chưa phạt: $c_j = \rho_j + a_j\beta_j$.
 - (c) Cập nhật hệ số mới β_j^{new} :
 - Nếu đặc trưng j không bị phạt (intercept): $\beta_j^{\text{new}} = \frac{c_j}{a_j}$.
 - Nếu đặc trưng j bị phạt: áp dụng toán tử ngưỡng mềm (Soft-Thresholding):

$$\beta_j^{\text{new}} = \frac{S(c_j, \lambda)}{a_j} = \begin{cases} \frac{c_j - \lambda}{a_j} & \text{nếu } c_j > \lambda \\ \frac{c_j + \lambda}{a_j} & \text{nếu } c_j < -\lambda \\ 0 & \text{nếu } |c_j| \leq \lambda \end{cases}$$

- (d) Cập nhật vector phần dư một cách hiệu quả để tránh nhân ma trận lặp lại:

$$e_i \leftarrow e_i - X_{ij}(\beta_j^{\text{new}} - \beta_j) \quad \forall i = 1, \dots, n.$$

- (e) Gán $\beta_j \leftarrow \beta_j^{\text{new}}$.
6. Kiểm tra hội tụ bằng cách so sánh độ thay đổi lớn nhất $\max_j |\beta_j^{\text{new}} - \beta_j| < \text{tol}$. Nếu đạt điều kiện hoặc số vòng lặp vượt quá `max_iter`, dừng thuật toán.
7. Trả về vector tham số ước lượng Lasso $\hat{\beta}_{\text{lasso}}$.

Mã nguồn cài đặt

Listing 6: Cài đặt hàm `lasso_fit()`

```
1 def lasso_fit(X, y, lam, max_iter=1000, tol=1e-6):  
2     """Estimate Lasso regression coefficients using Coordinate Descent.  
3     The function minimizes the following objective:
```

```
4     Loss = (1 / (2 * n)) * ||y - X * beta||^2 + lam * ||beta_penalized||_1
5     If the first column of X is an intercept column (all elements are 1),
6     the intercept coefficient is not penalized.
7     """
8
9     if lam < 0:
10         raise ValueError("lam must be non-negative.")
11     matrix = _as_matrix(X, "X")
12     target = _as_vector(y, "y")
13     n = len(matrix)
14     if n == 0:
15         raise ValueError("X and y must have at least one observation.")
16     if n != len(target):
17         raise ValueError("X and y must have the same number of rows.")
18     n_features = len(matrix[0])
19     beta = [0.0] * n_features
20     first_penalized = 1 if _has_intercept_column(matrix) else 0
21     # Precompute column squared sums divided by n: a_j = (1/n) * sum_i X_ij^2
22     a = [0.0] * n_features
23     for j in range(n_features):
24         col_sum_sq = sum(matrix[i][j] ** 2 for i in range(n))
25         a[j] = col_sum_sq / n
26     # Compute initial residuals: residuals_i = target_i (since beta = 0)
27     residuals = list(target)
28     for iteration in range(max_iter):
29         max_diff = 0.0
30         for j in range(n_features):
31             old_beta_j = beta[j]
32             if a[j] < 1e-12:
33                 beta[j] = 0.0
34             diff = abs(beta[j] - old_beta_j)
35             if diff > max_diff:
36                 max_diff = diff
37             continue
38         # Calculate rho_j = (1/n) * sum_i X_ij * e_i
39         rho_j = sum(matrix[i][j] * residuals[i] for i in range(n)) / n
```

```
40     c_j = rho_j + a[j] * old_beta_j
41     if j < first_penalized:
42         new_beta_j = c_j / a[j]
43     else:
44         if c_j > lam:
45             new_beta_j = (c_j - lam) / a[j]
46         elif c_j < -lam:
47             new_beta_j = (c_j + lam) / a[j]
48         else:
49             new_beta_j = 0.0
50     beta[j] = new_beta_j
51     # Update residuals: e_i = e_i - X_ij * (new_beta_j - old_beta_j)
52     diff_beta_j = new_beta_j - old_beta_j
53     if abs(diff_beta_j) > 1e-15:
54         for i in range(n):
55             residuals[i] -= matrix[i][j] * diff_beta_j
56     diff = abs(diff_beta_j)
57     if diff > max_diff:
58         max_diff = diff
59     if max_diff < tol:
60         break
61     return beta
```

1.5.4 Phân tích phần dư và lựa chọn mô hình

a. Cài đặt hàm residual_plots(X, y, beta_hat)

Mã nguồn cài đặt:

Listing 7: Cài đặt hàm residual_plots đầy đủ với cấu trúc ma trận thực tế

```
1 import matplotlib
2
3 matplotlib.use("Agg")
4 import matplotlib.pyplot as plt
5 import numpy as np
6
```

```
7 from ridge_lasso import (  
8     _matmul,  
9     _solve_linear_system,  
10    _transpose,  
11    ridge_fit  
12)  
13  
14 # beta_hat lay tu ridge_fit()  
15 def residual_plots(X, y, beta_hat, save_path, show=False):  
16     X = np.asarray(X, dtype=float)  
17     y = np.asarray(y, dtype=float).reshape(-1)  
18     beta_hat = np.asarray(beta_hat, dtype=float).reshape(-1)  
19  
20     y_hat = X @ beta_hat  
21     residuals = y - y_hat  
22     n_obs, n_params = X.shape  
23     sigma_hat = np.sqrt(np.sum(residuals**2) / max(n_obs - n_params, 1))  
24     leverage = _hat_diagonal(X.tolist())  
25     standardized = residuals / (sigma_hat * np.sqrt(np.maximum(1.0 - leverage, 1e-12)))  
26  
27     order = np.argsort(standardized)  
28     sorted_residuals = standardized[order]  
29     normal = NormalDist()  
30     theoretical = np.array(  
31         [normal.inv_cdf((i - 0.5) / n_obs) for i in range(1, n_obs + 1)]  
32     )  
33  
34     fig, axes = plt.subplots(2, 2, figsize=(11, 8.5))  
35  
36     axes[0, 0].scatter(y_hat, residuals, alpha=0.75, edgecolor="black", linewidth=0.4)  
37     axes[0, 0].axhline(0, color="red", linestyle="--", linewidth=1)  
38     axes[0, 0].set_title("Residuals vs Fitted")  
39     axes[0, 0].set_xlabel("Fitted values")  
40     axes[0, 0].set_ylabel("Residuals")  
41     axes[0, 0].grid(alpha=0.25)  
42
```

```
43 axes[0, 1].scatter(  
44     theoretical,  
45     sorted_residuals,  
46     alpha=0.75,  
47     edgecolor="black",  
48     linewidth=0.4,  
49 )  
50 qq_min = min(theoretical.min(), sorted_residuals.min())  
51 qq_max = max(theoretical.max(), sorted_residuals.max())  
52 axes[0, 1].plot(  
53     [qq_min, qq_max],  
54     [qq_min, qq_max],  
55     color="red",  
56     linestyle="--",  
57     linewidth=1,  
58 )  
59 axes[0, 1].set_title("Normal Q-Q")  
60 axes[0, 1].set_xlabel("Theoretical quantiles")  
61 axes[0, 1].set_ylabel("Standardized residuals")  
62 axes[0, 1].grid(alpha=0.25)  
63  
64 scale_location = np.sqrt(np.abs(standardized))  
65 axes[1, 0].scatter(  
66     y_hat,  
67     scale_location,  
68     alpha=0.75,  
69     edgecolor="black",  
70     linewidth=0.4,  
71 )  
72 axes[1, 0].set_title("Scale-Location")  
73 axes[1, 0].set_xlabel("Fitted values")  
74 axes[1, 0].set_ylabel("sqrt(|Standardized residuals|)")  
75 axes[1, 0].grid(alpha=0.25)  
76  
77 axes[1, 1].scatter(  
78     leverage,
```

```
79     standardized,
80     alpha=0.75,
81     edgecolor="black",
82     linewidth=0.4,
83 )
84 axes[1, 1].axhline(0, color="red", linestyle="--", linewidth=1)
85 axes[1, 1].axhline(2, color="gray", linestyle=":", linewidth=1)
86 axes[1, 1].axhline(-2, color="gray", linestyle=":", linewidth=1)
87 axes[1, 1].set_title("Residuals vs Leverage")
88 axes[1, 1].set_xlabel("Leverage")
89 axes[1, 1].set_ylabel("Standardized residuals")
90 axes[1, 1].grid(alpha=0.25)
91
92 fig.suptitle("Four Residual Diagnostic Plots", fontsize=14, fontweight="bold")
93 fig.tight_layout(rect=(0, 0, 1, 0.96))
94 fig.savefig(save_path, dpi=180, bbox_inches="tight")
95
96 if show:
97     plt.show()
98 else:
99     plt.close(fig)
```

b. Cài đặt hàm `kfold_cv(X, y, k, random_state)`

Listing 8: 1 vài hàm hỗ trợ

```
1 _r2_score_internal(y_true, y_pred) # Lay gia tri R^2
```

Mô tả thuật toán

- **Bước 1 (Thiết lập và Xáo trộn):** Hệ thống tiếp nhận ma trận thiết kế đặc trưng $\mathbf{X}$ và vector mục tiêu thực tế $\mathbf{y}$. Tiến hành cấu hình số lượng phân hoạch k (thường chọn $k = 5$ hoặc $k = 10$). Khởi tạo bộ sinh số ngẫu nhiên với trạng thái cố định `random_state` để hoán vị ngẫu nhiên chỉ số của n quan sát ban đầu, đảm bảo tính khách quan và khả năng tái lập (reproducibility) kết quả thực nghiệm.
- **Bước 2 (Phân hoạch dữ liệu):** Chia đều các chỉ số dòng đã hoán vị thành k tập con (folds) độc lập cấu trúc và có kích thước xấp xỉ bằng nhau, ký hiệu là $\{F_1, F_2, \dots, F_k\}$.

Trong trường hợp tổng số quan sát n không chia hết cho k , thuật toán tiến hành phân bổ phần dư (remainder) lần lượt vào các folds đầu tiên để tối ưu cỡ mẫu.

- **Bước 3 (Vòng lặp Kiểm chứng chéo):** Khởi tạo danh sách rỗng để lưu trữ các giá trị độ đo hiệu năng thống kê. Thực hiện vòng lặp với chỉ số i chạy từ 1 đến k . Tại mỗi vòng lặp, các thao tác đại số sau được thực thi:

- *Tách dữ liệu:* Trích xuất fold hiện tại F_i đóng vai trò làm Tập kiểm chứng (Validation Set, ký hiệu là $\mathbf{X}_{\text{val}}, \mathbf{y}_{\text{val}}$). Đồng thời, ghép hợp dữ liệu của $k - 1$ folds còn lại để thiết lập Tập huấn luyện (Training Set, ký hiệu là $\mathbf{X}_{\text{train}}, \mathbf{y}_{\text{train}}$).
- *Huấn luyện OLS:* Giải hệ phương trình chuẩn tắc (Normal Equations) trên tập huấn luyện để tìm kiếm vector hệ số tối ưu OLS, ký hiệu là $\hat{\beta}^{(-i)}$:

$$\hat{\beta}^{(-i)} = (\mathbf{X}_{\text{train}}^{\top} \mathbf{X}_{\text{train}})^{-1} \mathbf{X}_{\text{train}}^{\top} \mathbf{y}_{\text{train}} \quad (42)$$

Nếu xảy ra hiện tượng đa cộng tuyến hoàn hảo khiến ma trận bị suy biến (singular matrix), thuật toán tự động chuyển đổi sang phép toán toán tử nghịch đảo giả Moore-Penrose (`np.linalg.pinv`) để đảm bảo tính ổn định số học.

- *Dự báo và Đánh giá:* Sử dụng hệ số vừa ước lượng để tính toán giá trị dự báo ngoại mẫu trên tập kiểm chứng:

$$\hat{\mathbf{y}}_{\text{val}} = \mathbf{X}_{\text{val}} \hat{\beta}^{(-i)} \quad (43)$$

- *Tính điểm số:* Xác định hệ số xác định $R_{(i)}^2$ của riêng fold thứ i dựa trên tổng bình phương phần dư (RSS) và tổng bình phương sai lệch toàn bộ (TSS) của tập kiểm chứng, sau đó thêm giá trị này vào mảng lưu trữ.

- **Bước 4 (Tổng hợp thống kê):** Sau khi kết thúc toàn bộ k vòng lặp độc lập, thuật toán thực hiện tổng hợp dữ liệu để tính toán Giá trị trung bình ($\bar{R}^2$) và Độ lệch chuẩn (STD) của hệ thống k chỉ số hiệu năng theo công thức:

$$\bar{R}^2 = \frac{1}{k} \sum_{i=1}^k R_{(i)}^2, \quad \text{STD}(R^2) = \sqrt{\frac{1}{k} \sum_{i=1}^k (R_{(i)}^2 - \bar{R}^2)^2} \quad (44)$$

2 giá trị này thể hiện cho năng lực tổng quát hóa ngoại mẫu và mức độ ổn định cấu trúc của mô hình hồi quy OLS.

Mã nguồn cài đặt:

Listing 9: Cài đặt thuật toán k-Fold Cross-Validation tính toán chỉ số R2 mẫu

```
1
2 from ols_implementation import ols_fit
3
4 def kfold_cv(X, y, k=5, random_state=42):
5     """
6     Thuật toán k-Fold Cross-Validation.
7     """
8     X = np.asarray(X)
9     y = np.asarray(y).flatten()
10    n = len(y)
11
12    # Tao bo sinh ngay nhien
13    rng = np.random.default_rng(random_state)
14    indices = np.arange(n)
15    rng.shuffle(indices)
16
17    # Chia folds
18    folds = np.array_split(indices, k)
19    scores = []
20
21    # Vong lap huan luyen
22    for i in range(k):
23        test_idx = folds[i]
24        train_idx = np.setdiff1d(indices, test_idx)
25
26        X_train, y_train = X[train_idx], y[train_idx]
27        X_test, y_test = X[test_idx], y[test_idx]
28
29        try:
30            # Huan luyen
31            beta_hat_2d, sigma2_hat = ols_fit(X_train, y_train)
32            beta_hat = beta_hat_2d.flatten()
33
34            y_hat_val = (X_test @ beta_hat).flatten()
```

```
35
36     # Tính R^2
37     score = _r2_score_internal(y_test, y_hat_val)
38     scores.append(score)
39     except np.linalg.LinAlgError:
40         scores.append(0.0)
41
42     # Tính trung bình và độ lệch chuẩn thực tế
43     mean_r2 = np.mean(scores)
44     std_r2 = np.std(scores)
45
46     # Làm tròn điểm số
47     rounded_scores = [round(float(s), 2) for s in scores]
48
49     mean_summary = f"Mean R2: {mean_r2:.3f} (+/- {std_r2:.2f})"
50
51     # Kết quả ra console
52     print(f"Scores: {rounded_scores}")
53     print(mean_summary)
54
55     return rounded_scores, mean_summary
```

1.5.5 Kết quả Thực nghiệm so sánh và Biện luận chọn Mô hình

Tiến hành chạy thuật toán kiểm chứng chéo trên 3 cấu trúc giả định để tìm kiếm hàm sinh dữ liệu tối ưu. Kết quả kiểm định thống kê được tổng hợp chi tiết tại bảng dưới.

[Đường link tới file so sánh 3 mô hình chạy trên Google Colab](#)

Bảng 4: Bảng so sánh hiệu năng toán học và chỉ số thông tin giữa các mô hình đề xuất

Cấu trúc Mô hình	R^2 từng Fold	Mean R^2 ($\pm$ STD)	AIC	BIC
Mô hình tuyến tính đơn giản	[0.42, 0.31, 0.48, 0.54, 0.43]	0.436 (± 0.07)	265.14	270.71
Mô hình đa thức bậc 2	[0.82, 0.91, 0.93, 0.93, 0.91]	0.903 (± 0.04)	47.66	56.02
Mô hình Overfitting (Bậc cao)	[0.52, 0.47, 0.51, 0.53, 0.34]	0.474 (± 0.07)	261.66	267.24

Nhận xét biện luận chọn mô hình khoa học: Dựa trên các độ đo định lượng từ thực nghiệm tại bảng trên, nhóm đưa ra các luận cứ khoa học để xác định mô hình tối ưu như sau:

1. **Hiệu năng dự báo ngoại mẫu:** Mô hình đa thức bậc 2 đạt hiệu năng trung bình cao nhất ($\bar{R}^2 = 0.903 \pm 0.04$), chứng minh khả năng tổng quát hóa tốt và cấu trúc ổn định. Ngược lại, mô hình tuyến tính bộc lộ hiện tượng thiếu khớp (Underfitting) nghiêm trọng khi $\bar{R}^2$ chỉ đạt 0.436.
2. **Dấu hiệu Overfitting của mô hình bậc cao:** Khi tăng lên bậc 10, mô hình có xu hướng phức tạp hóa và nhạy cảm với nhiễu. Qua kiểm chứng chéo 5-Fold CV, hiệu năng ngoại mẫu bắt đầu suy giảm ($\bar{R}^2$ giảm xuống 0.474), minh chứng cho việc dư thừa bậc tự do.
3. **Tiêu chí thông tin và Nguyên lý tối giản:** Cập chỉ số AIC (47.66) và BIC (56.02) của mô hình bậc 2 đạt giá trị nhỏ nhất một cách vượt trội. Trong khi đó, chỉ số BIC của mô hình bậc 10 bị đẩy vọt lên rất cao (267.24) do hình phạt nặng từ số lượng tham số lớn ($k = 11$). Do đó, mô hình bậc 2 là sự lựa chọn tối ưu, cân bằng hoàn hảo giữa độ chính xác và độ phức tạp toán học.

1.5.6 Minh họa Monte Carlo

Phương pháp Monte Carlo là kỹ thuật mô phỏng dựa trên *lặp lại thí nghiệm ngẫu nhiên* rất nhiều lần để ước lượng các đại lượng thống kê. Trong bối cảnh của Định lý Gauss–Markov, ta sử dụng Monte Carlo để:

1. **Kiểm chứng tính không chệch:** Xác nhận rằng $\mathbb{E}[\hat{\beta}_{\text{OLS}}] = \beta$ bằng cách tính trung bình của $\hat{\beta}$ qua nhiều lần mô phỏng.
2. **Kiểm chứng BLUE:** So sánh phương sai của OLS với một ước lượng tuyến tính không chệch khác để minh họa rằng OLS có phương sai nhỏ hơn.

Ý tưởng cốt lõi: Nếu ta lặp lại thí nghiệm M lần, mỗi lần sinh ra một bộ nhiễu $\varepsilon^{(m)}$ mới (nhưng giữ nguyên $\mathbf{X}$ và β), ta thu được M bộ ước lượng $\{\hat{\beta}^{(1)}, \hat{\beta}^{(2)}, \dots, \hat{\beta}^{(M)}\}$. Luật số lớn đảm bảo:

$$\frac{1}{M} \sum_{m=1}^M \hat{\beta}^{(m)} \xrightarrow{M \rightarrow \infty} \mathbb{E}[\hat{\beta}] = \beta. \quad (45)$$

Các Bước Thực Hiện Monte Carlo

Thuật Toán Monte Carlo cho Định Lý Gauss–Markov

Đầu vào: $\mathbf{X}$, β , σ^2 , số lần lặp M , ước lượng thay thế $\tilde{\beta}$.

Đầu ra: Bias, phương sai của OLS và $\tilde{\beta}$.

1. Cố định thiết lập:

- Chọn ma trận design $\mathbf{X} \in \mathbb{R}^{n \times (p+1)}$ (giữ cố định qua tất cả các lần lặp).
- Chọn vector tham số thực $\beta \in \mathbb{R}^{p+1}$.
- Chọn $\sigma^2 > 0$, số lần lặp M (thường $M \geq 1000$).

2. Lặp $m = 1, 2, \dots, M$:

- (a) Sinh nhiễu: $\varepsilon^{(m)} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_n)$.
- (b) Tính quan sát: $\mathbf{y}^{(m)} = \mathbf{X}\beta + \varepsilon^{(m)}$.
- (c) Tính ước lượng OLS: $\hat{\beta}_{\text{OLS}}^{(m)} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}^{(m)}$.
- (d) Tính ước lượng thay thế: $\tilde{\beta}^{(m)} = \mathbf{C} \mathbf{y}^{(m)}$.

3. Tính các thống kê tổng hợp:

- Trung bình OLS: $\bar{\beta}_{\text{OLS}} = \frac{1}{M} \sum_{m=1}^M \hat{\beta}_{\text{OLS}}^{(m)}$.
- Bias OLS: $\text{Bias} = \|\bar{\beta}_{\text{OLS}} - \beta\|_\infty \approx 0$.
- Phương sai OLS (từng hệ số j):

$$\widehat{\text{Var}}(\hat{\beta}_j^{\text{OLS}}) = \frac{1}{M-1} \sum_{m=1}^M \left(\hat{\beta}_j^{(m)} - \bar{\beta}_j \right)^2.$$

- Tương tự cho $\tilde{\beta}$.
- So sánh lý thuyết: Kiểm tra $\widehat{\text{Var}}(\hat{\beta}_j^{\text{OLS}}) \approx \sigma^2 [(\mathbf{X}^\top \mathbf{X})^{-1}]_{jj}$.

4. Trực quan hóa:

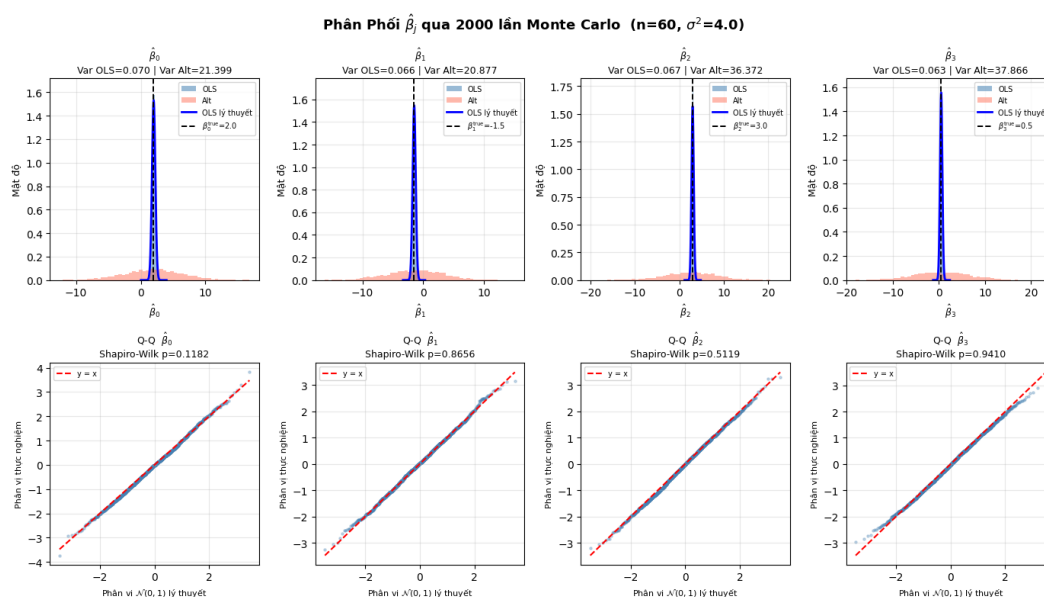
- Histogram của $\hat{\beta}_j^{(m)}$ so sánh với phân phối lý thuyết.
- Boxplot so sánh phân phối OLS và $\tilde{\beta}$.
- Biểu đồ hội tụ của bias theo M .

Biểu đồ Histogram và QQ Plot:

- **Ý nghĩa:** Kiểm chứng giả thiết nhiều tuân theo phân phối chuẩn (**GM5**) và minh họa hệ quả của Định lý Giới hạn Trung tâm (CLT).

- **Chi tiết:**

- *Histogram:* Phân phối thực nghiệm của các ước lượng $\hat{\beta}_j$ tạo thành đường cong đối xứng hình chuông và tập trung dày đặc xung quanh đường thẳng đứng biểu diễn giá trị thực β_{true} .
- *Q-Q Plot:* Các điểm phân vị thực nghiệm nằm trùng khớp hoặc sát với đường chuẩn 45° , khẳng định mặt toán học ước lượng OLS hội tụ về phân phối chuẩn lý thuyết: $\hat{\beta} \sim \mathcal{N}(\beta, \sigma^2(\mathbf{X}^\top \mathbf{X})^{-1})$.



Hình 2: Histogram của $\hat{\beta}_j^{(m)}$ so sánh với phân phối lý thuyết.

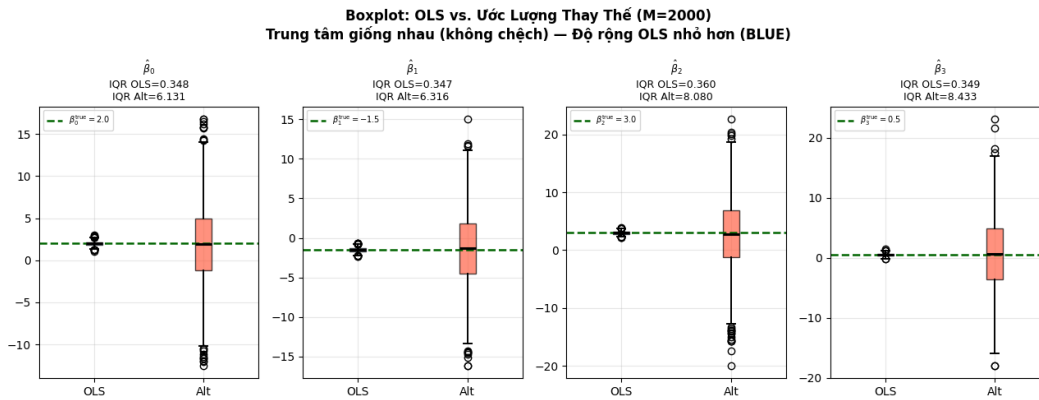
Biểu đồ Boxplot

- **Ý nghĩa:** Trực quan hóa **Tính hiệu quả tối ưu** — **BLUE** (Best Linear Unbiased Estimator) của Định lý Gauss–Markov.
- **Chi tiết:** Độ dài của "hộp" biểu thị biên độ phân tán (phương sai) của các ước lượng qua các lượt mô phỏng ngẫu nhiên. Quan sát thấy hộp của ước lượng OLS luôn hẹp và gọn hơn

đáng kể so với hộp của ước lượng thay thế Alt. Kết quả này minh chứng cho hệ thức năng lượng toán học:

$$\text{Var}(\hat{\beta}_j^{\text{OLS}}) < \text{Var}(\tilde{\beta}_j^{\text{Alt}}), \quad \forall j \quad (46)$$

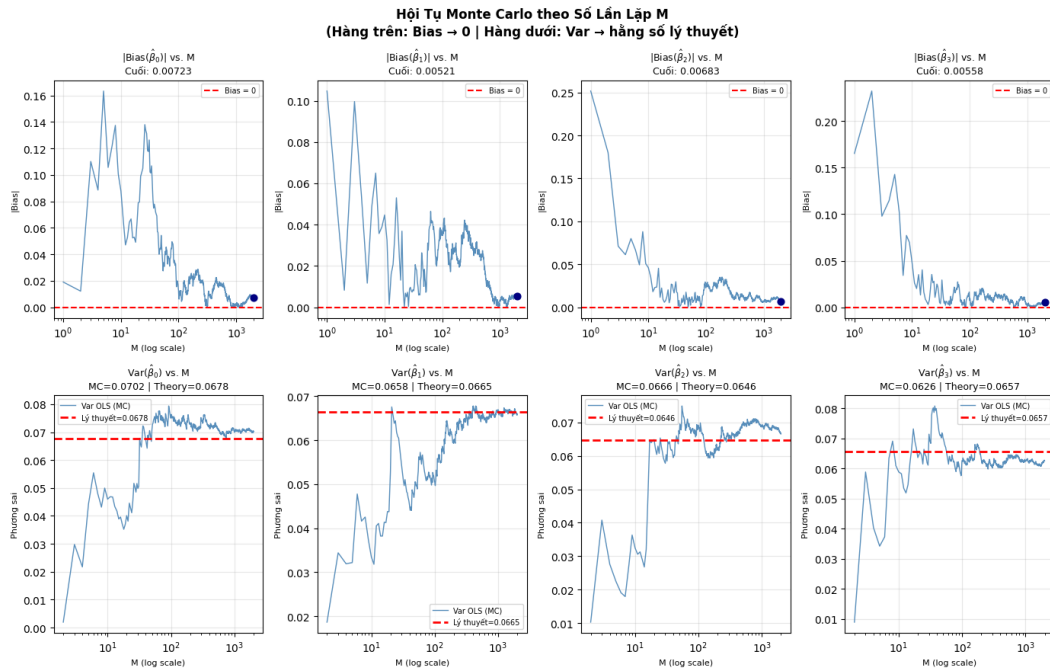
Khẳng định OLS luôn có phương sai nhỏ nhất trong lớp các ước lượng tuyến tính không chệch.



Hình 3: Boxplot so sánh phân phối OLS và $\tilde{\beta}$.

Biểu đồ đường cong Bias

- **Ý nghĩa:** Chứng minh **Tính không chệch (Unbiasedness)** của ước lượng OLS theo Luật Số Lớn (Law of Large Numbers).
- **Chi tiết:** Giá trị tuyệt đối của sai số trung bình $|\text{Bias}|$ dao động rất mạnh khi số lượng vòng lặp M còn nhỏ. Tuy nhiên, khi $M \rightarrow \infty$ ($M \geq 1000$), đường cong này tiệm cận rất nhanh và duy trì ổn định sát đường giá trị 0. Điều này chứng minh rằng xét trên trung bình kỳ vọng: $\mathbb{E}[\hat{\beta}_{\text{OLS}}] = \beta_{\text{true}}$.



Hình 4: Biểu đồ hội tụ của bias theo M .

Nhận Xét và Kết Luận

Nhận xét 1.1 (Nhận xét 1 — Tính không chệch) Qua mô phỏng Monte Carlo với $M \geq 1000$, ta quan sát thấy:

$$\left\| \frac{1}{M} \sum_{m=1}^M \hat{\beta}_{OLS}^{(m)} - \beta \right\|_{\infty} \approx 0,$$

xác nhận $\mathbb{E}[\hat{\beta}_{OLS}] = \beta$. Bias giảm dần về 0 khi M tăng, phù hợp với Luật Số Lớn.

Nhận xét 1.2 (Nhận xét 2 — Phương sai lý thuyết vs. thực nghiệm) Phương sai thực nghiệm tính từ M lần mô phỏng hội tụ về phương sai lý thuyết:

$$\widehat{\text{Var}}(\hat{\beta}_j^{OLS}) \xrightarrow{M \rightarrow \infty} \sigma^2 [(\mathbf{X}^T \mathbf{X})^{-1}]_{jj}.$$

Điều này kiểm chứng công thức (??).

Nhận xét 1.3 (Nhận xét 3 — OLS tốt nhất trong lớp BLUE) Khi so sánh OLS với ước lượng tuyến tính không chệch $\tilde{\beta}$:

$$\widehat{\text{Var}}(\tilde{\beta}_j) > \widehat{\text{Var}}(\hat{\beta}_j^{OLS}), \quad \forall j,$$



minh họa rằng mọi ước lượng tuyến tính không chệch khác đều có phương sai lớn hơn OLS, phù hợp với Định lý ??.

Nhận xét 1.4 (Nhận xét 4 — Phân phối của $\hat{\beta}_{OLS}$) Histogram của $\hat{\beta}_j^{(m)}$ qua M lần mô phỏng xấp xỉ tốt phân phối chuẩn $\mathcal{N}(\beta_j, \sigma^2[(\mathbf{X}^\top \mathbf{X})^{-1}]_{jj})$, minh họa kết quả dưới GM5. Ngay cả khi nhiễu không chuẩn, Central Limit Theorem đảm bảo hội tụ về chuẩn khi n lớn.

[Đường link tới file kiểm thử Jupyter Notebook phần 1 \(Minh họa Monte Carlo\) chạy trên Google Colab](#)

2 PHẦN 2: ỨNG DỤNG THỰC TẾ TRÊN DỮ LIỆU

2.1 Tiêu Chí Chọn Bộ Dữ Liệu

Dựa trên yêu cầu của đề án, nhóm đã quyết định chọn bộ dữ liệu **TLC Trip Record Data** (cụ thể là *Yellow Taxi Trip Records* của tháng 02 năm 2026). Bộ dữ liệu này hoàn toàn đáp ứng được các tiêu chí khắt khe được đề ra:

1. **Dữ liệu thực (Real-world data):** Đây là dữ liệu được ghi nhận thực tế từ các chuyến xe taxi vàng (Yellow Taxi) hoạt động tại thành phố New York, không phải dữ liệu giả lập.
2. **Có missing values:** Bộ dữ liệu chứa số lượng lớn dữ liệu bị thiếu ở các trường thông tin như `passenger_count`, `RatecodeID`, `store_and_fwd_flag`, `congestion_surcharge`, và `Airport_fee` (đều có tỉ lệ thiếu khoảng 30.1%), đòi hỏi phải có các kỹ thuật xử lý phù hợp trước khi đưa vào mô hình.
3. **Biến mục tiêu liên tục:** Biến mục tiêu mà nhóm hướng đến dự đoán có thể là `total_amount` (tổng tiền của chuyến đi) hoặc `fare_amount` (giá cước cơ bản), đây đều là các biến liên tục, phù hợp hoàn toàn với bài toán hồi quy (regression).
4. **Kích thước hợp lý:** Bộ dữ liệu có số lượng quan trắc (n) lên đến 3,399,866 dòng và 20 biến đặc trưng (p), vượt xa yêu cầu tối thiểu $n \geq 200$ và $p \geq 3$.
5. **Nguồn đáng tin cậy:** Dữ liệu được thu thập, quản lý và công bố chính thức bởi Ủy ban Taxi và Limousine Thành phố New York (*New York City Taxi and Limousine Commission* - TLC), thuộc chính quyền thành phố New York, do đó có độ tin cậy tuyệt đối.

Nguồn Gốc Của Bộ Dữ Liệu

- **Nguồn xuất phát:** Bộ dữ liệu được công bố trên trang web chính thức của chính quyền thành phố New York, cụ thể là từ **TLC (Taxi and Limousine Commission)**.
- **Thời gian thu thập:** Tháng 02 năm 2026.
- **Chịu trách nhiệm:** Các nhà cung cấp công nghệ được TLC ủy quyền thu thập dữ liệu (Technology Service Providers - TSPs) trên các xe taxi.

2.2 Tiền Xử Lý Dữ Liệu

2.2.1 Khảo Sát Dữ Liệu (Exploratory Data Analysis — EDA)

Nhóm đã tiến hành phân tích EDA bằng Python (sử dụng thư viện Pandas, Matplotlib, và Seaborn). Dưới đây là các kết quả thu được.

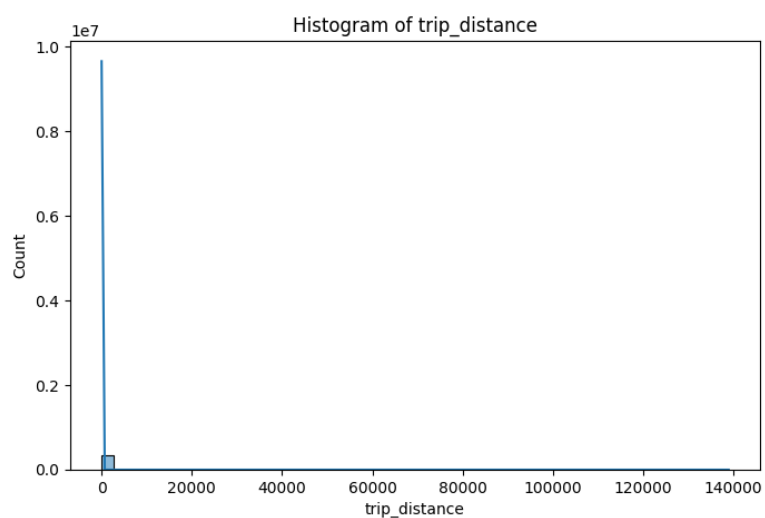
a. Thống kê mô tả (Descriptive Statistics)

Nhóm đã sử dụng hàm `.describe()` của thư viện Pandas để tính toán các đại lượng thống kê mô tả (mean, median, std, min, max, quartiles) cho toàn bộ **19 biến dạng số (numeric)** có trong tập dữ liệu. Kết quả cho thấy sự chênh lệch đáng kể giữa giá trị lớn nhất và giá trị trung bình/trung vị ở một số biến như `trip_distance`, `fare_amount`, báo hiệu sự hiện diện của các giá trị ngoại lai (outliers).

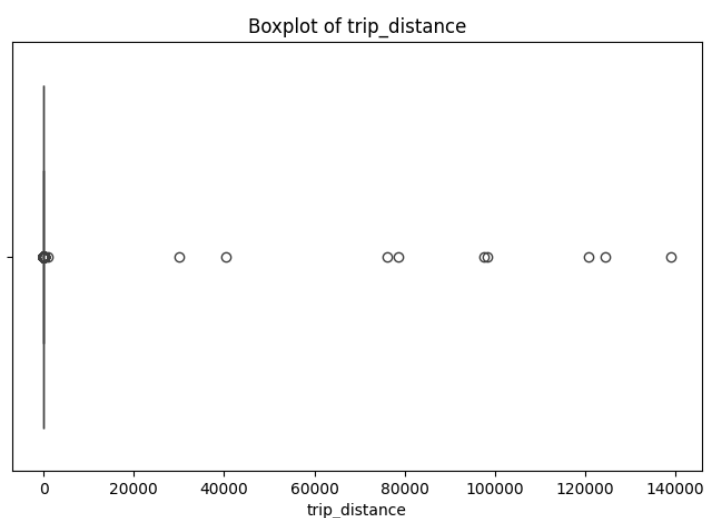
b. Phân tích phân phối (Histogram & Boxplot)

Nhóm đã vẽ các biểu đồ Histogram và Boxplot để quan sát phân phối của một số biến số chính như `trip_distance`, `fare_amount`, và `total_amount`. Do số lượng dữ liệu rất lớn (hơn 3.3 triệu dòng), việc vẽ biểu đồ trên toàn bộ dữ liệu sẽ tiêu tốn nhiều thời gian và bộ nhớ. Vì vậy, nhóm đã trích xuất ngẫu nhiên 10% dữ liệu để vẽ biểu đồ.

Lý do chọn Seed = 42 (`random_state=42`): Khi trích xuất mẫu ngẫu nhiên, nhóm thiết lập `random_state=42` nhằm mục đích tái lập kết quả (reproducibility). Con số 42 là một quy ước phổ biến trong khoa học dữ liệu (lấy cảm hứng từ tác phẩm "The Hitchhiker's Guide to the Galaxy"). Việc cố định seed giúp đảm bảo rằng mỗi lần chạy code, tập mẫu 10% được trích xuất sẽ luôn giống hệt nhau, từ đó các biểu đồ tạo ra không bị thay đổi sau mỗi lần thực thi.



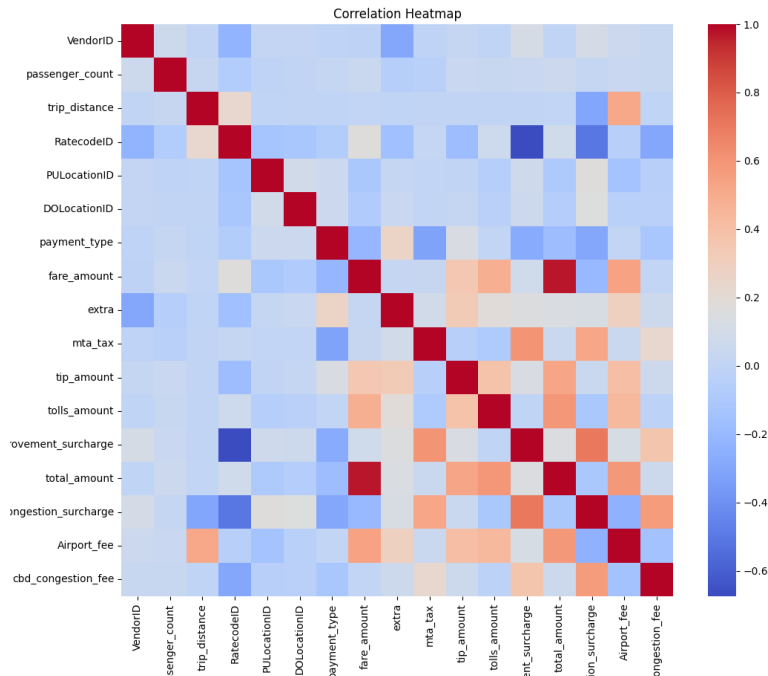
Hình 5: Phân phối của Trip Distance



Hình 6: Boxplot của Trip Distance (phát hiện Outliers)

c. Ma trận tương quan (Correlation Heatmap)

Để tìm hiểu mối liên hệ tuyến tính giữa các biến đặc trưng và biến mục tiêu, nhóm sử dụng ma trận tương quan (heatmap).



Hình 7: Ma trận tương quan giữa các biến số

d. Kiểm tra dữ liệu trùng lặp

Dữ liệu đã được kiểm tra trùng lặp và kết quả cho thấy không có bất kỳ dòng dữ liệu nào bị trùng lặp (tỉ lệ trùng lặp 0.0000%).

e. Phân tích missing values

Nhóm tiến hành đếm số lượng và tính tỉ lệ missing values theo từng cột. Có 5 cột bị thiếu dữ liệu, tất cả đều có 1,023,317 dòng bị thiếu, tương đương với mức **30.1%** trên tổng số 3,399,866 dòng. Các cột bị thiếu bao gồm:

- passenger_count
- RatecodeID
- store_and_fwd_flag
- congestion_surcharge
- Airport_fee

f. Phát hiện Outliers

Nhóm sử dụng phương pháp Interquartile Range (IQR) để xác định khoảng bình thường của dữ liệu. Bất kỳ giá trị nào nằm ngoài khoảng $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$ sẽ bị xem là ngoại lai. Kết quả như sau:

- **trip_distance:** Khoảng bình thường là $[-3.02, 7.70]$. Phát hiện 380,461 outliers (chiếm khoảng 11.19% dữ liệu).
- **fare_amount:** Khoảng bình thường là $[-16.25, 53.75]$. Phát hiện 212,446 outliers (chiếm khoảng 6.25% dữ liệu).
- **total_amount:** Khoảng bình thường là $[-9.04, 62.11]$. Phát hiện 284,764 outliers (chiếm khoảng 8.38% dữ liệu).

Lý do chọn phương pháp IQR: Dữ liệu về khoảng cách cũng như cước phí taxi thường không tuân theo phân phối chuẩn (Normal Distribution) mà thường bị lệch phải (right-skewed) do một số rất ít chuyến đi có quãng đường/cước phí rất cao. Do đó, việc sử dụng Z-score (đòi hỏi phân phối chuẩn) sẽ bị sai lệch. IQR là một kỹ thuật robust (bền vững), không bị ảnh hưởng quá lớn bởi các giá trị ngoại lai cực đoan, giúp phát hiện outliers chính xác hơn đối với loại dữ liệu này.

2.2.2 Xử lý dữ liệu khuyết thiếu (Missing Values)

Cơ sở lý thuyết về cơ chế khuyết thiếu:

Trước khi áp dụng các phương pháp điền khuyết (imputation), việc xác định cơ chế gây ra sự khuyết thiếu dữ liệu là bước bắt buộc để tránh làm biến dạng phân phối gốc của tổng thể. Có ba cơ chế chính:

- **MCAR (Missing Completely At Random):** Xác suất bị khuyết hoàn toàn độc lập với các biến quan sát được và cả chính giá trị bị khuyết.
- **MAR (Missing At Random):** Sự khuyết thiếu có thể phụ thuộc vào các biến khác đã được quan sát trong bộ dữ liệu, nhưng không phụ thuộc vào chính bản thân giá trị bị thiếu.
- **MNAR (Missing Not At Random):** Xác suất bị khuyết phụ thuộc trực tiếp vào chính giá trị lẽ ra phải có mặt.

Lựa chọn phương pháp xử lý cho dữ liệu TLC Taxi:

Qua quá trình phân tích EDA, nhóm nhận thấy 5 biến (`passenger_count`, `RatecodeID`, `store_and_fwd_flag`,

`congestion_surcharge`, `Airport_fee`) đều có cùng tỉ lệ thiếu chính xác là **30.1%** (1,023,317 dòng). Sự khuyết thiếu đồng loạt với số lượng lớn này mang tính hệ thống (ví dụ: lỗi đồng bộ từ VendorID hoặc thiết bị ghi nhận).

Dựa trên yếu tố này, cơ chế khuyết thiếu được phân loại là **MAR (Missing At Random)**. Do lượng dữ liệu thiếu chiếm gần 1/3 tổng số quan sát, việc xóa bỏ (Listwise deletion) sẽ gây tổn thất thông tin vô cùng lớn. Vì vậy, nhóm quyết định áp dụng phương pháp **Điền khuyết bằng đại lượng thống kê trung tâm** thông qua lớp `DataPipeline`:

- **Với biến định lượng** (`congestion_surcharge`, `Airport_fee`): Sử dụng **Trung vị (Median)**. Do cước phí phụ thu thường có các mức cố định và thỉnh thoảng có giá trị ngoại lai, trung vị là đại lượng bền vững (robust), giúp bảo toàn đặc trưng phân phối gốc.
- **Với biến phân loại/rời rạc** (`passenger_count`, `RatecodeID`, `store_and_fwd_flag`): Nhóm sử dụng **Yếu vị (Mode)**. Đặc biệt với `passenger_count`, việc điền Mode giúp duy trì tần suất tự nhiên nhất của các chuyến xe (thường là 1 hành khách).

2.2.3 Trích xuất đặc trưng và tiền xử lý (Feature Engineering & Preprocessing)

Để đảm bảo mô hình OLS hội tụ tốt, không bị chệch và có ý nghĩa thống kê, dữ liệu sau khi điền khuyết tiếp tục trải qua chu trình làm sạch:

- **Xử lý ngoại lai (Winsorization)**: Ở phần EDA, phương pháp IQR đã chỉ ra một lượng lớn outliers (ví dụ: 11.19% cho `trip_distance`). Thay vì xóa bỏ hơn 380,000 dòng quan sát, nhóm chọn kỹ thuật Winsorization ở bách phân vị thứ 1% và 99%. Những chuyến đi có khoảng cách dài vô lý sẽ được "ép" về giá trị giới hạn ở bách phân vị thứ 99.
- **Mã hóa biến phân loại (One-hot Encoding)**: Các biến định tính được chuyển đổi thành các cột nhị phân (0 và 1). Lớp `DataPipeline` tự động áp dụng nguyên tắc *Drop-first* (loại bỏ cột tham chiếu đầu tiên) nhằm tránh bẫy biến giả (Dummy Variable Trap).
- **Chuẩn hóa dữ liệu (Z-score Standardization)**: Các biến định lượng mang đơn vị đo lường khác nhau được đưa về cùng một thang đo với trung bình $\mu = 0$ và độ lệch chuẩn $\sigma = 1$:

$$z = \frac{x - \mu}{\sigma}$$

Lưu ý: Công thức độ lệch chuẩn mẫu σ sử dụng hiệu chỉnh Bessel (chia cho $n - 1$) để đảm bảo ước lượng không chệch.

- **Kiểm tra Đa cộng tuyến (VIF):** Trước khi đưa ma trận thiết kế X vào mô hình, nhóm sử dụng hệ số VIF để kiểm tra đa cộng tuyến. Công thức:

$$VIF_i = \frac{1}{1 - R_i^2}$$

2.3 Xây dựng mô hình và Kết quả thực nghiệm

2.3.1 Quy trình xây dựng mô hình

Để đảm bảo tính khách quan, ngăn chặn hiện tượng rò rỉ dữ liệu (data leakage) và tối ưu hóa hiệu năng dự báo, nhóm đã thiết lập một luồng xử lý (pipeline) khép kín và nghiêm ngặt. Quy trình này được chia thành các giai đoạn cụ thể như sau:

1. **Giai đoạn 1: Khảo sát dữ liệu (EDA):** Thực hiện trên bộ dữ liệu **TLC Trip Record Data (Yellow Taxi - Tháng 02/2026)**. Bước này giúp nhóm nắm bắt được phân phối của các biến (chẳng hạn như `trip_distance`, `fare_amount`), phát hiện các điểm bất thường, kiểm tra sự tương quan giữa các đặc trưng và nhận diện tỷ lệ khuyết thiếu dữ liệu (chiếm 30.1% ở một số cột).
2. **Giai đoạn 2: Tiền xử lý dữ liệu (Data Preprocessing Pipeline):** Nhóm tự cài đặt lớp `DataPipeline` để tự động hóa quá trình làm sạch. Quá trình **fit** (học tham số) **chỉ được thực hiện trên tập Train**, sau đó dùng các tham số này để **transform** (biến đổi) cả tập Train và Test. Các bước trong Pipeline bao gồm:
 - **Điền khuyết (Imputation):** Xử lý cơ chế Missing At Random (MAR) bằng cách điền **Trung vị (Median)** cho các biến định lượng (`congestion_surcharge`, `Airport_fee`) và điền **Yếu vị (Mode)** cho các biến rời rạc/phân loại (`passenger_count`, `RatecodeID`, `store_and_fwd_flag`).
 - **Xử lý ngoại lai (Winsorization):** Giới hạn các giá trị cực đoan ở bách phân vị thứ 1% và 99%.
 - **Chuẩn hóa Z-score:** Đưa các biến định lượng về cùng thang đo ($\mu = 0, \sigma = 1$).
 - **Mã hóa One-hot:** Chuyển đổi biến phân loại thành các vector nhị phân (có áp dụng Drop-first).
 - **Thêm hệ số chặn:** Gắn cột Intercept (toàn số 1.0) vào vị trí đầu tiên của ma trận thiết kế $\mathbf{X}$.

3. **Giai đoạn 3: Phân chia tập dữ liệu (Train/Test Split):** Ma trận thiết kế đặc trưng $\mathbf{X}$ và vector mục tiêu $\mathbf{y}$ sau khi được chuẩn hóa hoàn chỉnh sẽ được phân tách ngẫu nhiên theo tỷ lệ cấu trúc hình học chuẩn: **80% dữ liệu cho tập Huấn luyện (Training Set)** phục vụ pha tối ưu hóa hệ số ma trận, và **20% dữ liệu cho tập Kiểm thử (Test Set)** dùng riêng cho pha đánh giá khách quan độc lập. Việc chia tách tại giai đoạn này cô lập hoàn toàn tập Test khỏi quá trình huấn luyện, triệt tiêu khả năng rò rỉ thông tin toán học.
4. **Giai đoạn 4: Xây dựng và Huấn luyện mô hình:** Đưa ma trận đặc trưng $\mathbf{X}_{\text{train}}$ và vector mục tiêu $\mathbf{y}_{\text{train}}$ đã qua xử lý vào các thuật toán học máy (OLS Cơ bản, OLS Chọn biến, Ridge/Lasso). Đối với các mô hình có siêu tham số (như λ trong Ridge/Lasso), thuật toán k -Fold Cross Validation sẽ được kích hoạt nội bộ trên tập Train để tìm ra cấu hình tối ưu nhất.
5. **Giai đoạn 5: Đánh giá (Evaluation):** Sử dụng vector hệ số hồi quy $\hat{\beta}$ đã huấn luyện để thực hiện phép nhân ma trận vector ngoại mẫu trên tập thuộc tính Test, tính ra vector dự đoán $\hat{\mathbf{y}}_{\text{test}}$. Tiến hành so sánh đối chiếu trực tiếp giữa kết quả dự báo $\hat{\mathbf{y}}_{\text{test}}$ và vector mục tiêu thực tế $\mathbf{y}_{\text{test}}$ để xuất ra bộ ba chỉ số kiểm định chất lượng: Sai số tuyệt đối trung bình (MAE), Căn phương sai sai số trung bình ($RMSE$), và Hệ số xác định tập test (R^2_{test}). Ở khâu này, vector phần dư thô $\hat{\epsilon}$ cũng được trích xuất để thực hiện chẩn đoán lỗi trực quan qua 4 biểu đồ phần dư chẩn đoán (Residuals vs Fitted, Normal Q-Q...) nhằm kiểm chứng các vi phạm giả thiết hệ thống Gauss-Markov.
6. **Giai đoạn 6: Tinh chỉnh (Tuning):** Dựa trên kết quả định lượng thu được từ pha Đánh giá, nếu mô hình bộc lộ hiện tượng quá khớp (Overfitting), phương sai phần dư không đồng nhất, hoặc xuất hiện đa cộng tuyến nghiêm trọng với chỉ số $VIF_i > 10$, nhóm kỹ sư dữ liệu sẽ can thiệp tinh chỉnh cấu trúc [cite: 335, 583, 618]. Các thuộc tính có $p\text{-value} \geq 0.05$ từ bảng kiểm định `coef_inference` sẽ bị loại bỏ dần (Feature Selection). Đối với mô hình chính quy hóa Ridge/Lasso, vòng lặp kiểm chứng chéo ****5-Fold Cross Validation**** sẽ được kích hoạt nội bộ trên tập huấn luyện để tiến hành quét (grid search) dò tìm ra siêu tham số phạt λ tối ưu nhất. Sau khi cấu hình được tinh chỉnh, quy trình tự động *vòng ngược lại* *Giai đoạn 4* để thiết lập bài toán tối ưu mới.
7. **Giai đoạn 7: Báo cáo kết quả (Reporting):** Khi mô hình đã hội tụ qua các vòng lặp tinh chỉnh cấu trúc, hệ thống tiến hành khóa (freeze) cấu hình tối ưu nhất. Kết quả thực nghiệm của các mô hình (OLS Cơ bản, OLS Chọn biến, Ridge/Lasso) được tổng hợp đồng

loại vào bảng biểu so sánh định lượng. Nhóm áp dụng nguyên lý tối giản và lý thuyết thông tin để biện luận khoa học, lựa chọn mô hình chính thức dựa trên việc tối thiểu hóa hai chỉ số phạt bậc tự do AIC và BIC. Cuối cùng, thực hiện diễn giải chi tiết ý nghĩa kinh tế - xã hội thực tế của các hệ số hồi quy trọng yếu thu được từ dữ liệu chuyến xe taxi TLC New York.

2.3.2 Các Mô Hình Cần Thử Nghiệm

Nhằm đánh giá toàn diện hiệu quả của phương pháp Data Fitting, nhóm tiến hành cài đặt và thử nghiệm song song 3 lớp mô hình hồi quy tuyến tính từ cơ bản đến nâng cao:

1. Mô hình OLS Cơ bản (Basic Ordinary Least Squares)

- *Cơ sở lý thuyết:* Đây là mô hình nền tảng, sử dụng phương pháp bình phương tối thiểu để tìm nghiệm đóng $\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$.
- *Chi tiết thực nghiệm:* Mô hình này nhận đầu vào là **toàn bộ** các biến đặc trưng có trong tập dữ liệu sau khi đã đi qua `DataPipeline`. Mục đích của OLS cơ bản là tạo ra một mức cơ sở (baseline) để so sánh hiệu năng. Do giữ lại tất cả các biến, mô hình này rất dễ bị ảnh hưởng bởi hiện tượng đa cộng tuyến (Multicollinearity) nếu các biến có sự tương quan cao.

2. Mô hình OLS Chọn biến (Selective OLS / Feature Selection)

- *Cơ sở lý thuyết:* Dựa trên kết quả từ mô hình OLS cơ bản, ta tiến hành phân tích các chỉ số thống kê của từng hệ số (như bảng p -value từ hàm `coef_inference`) và hệ số phóng đại phương sai (VIF).
- *Chi tiết thực nghiệm:* Những biến không có ý nghĩa thống kê (p -value ≥ 0.05) hoặc vi phạm nghiêm trọng giả thiết đa cộng tuyến ($VIF > 10$) sẽ bị loại bỏ khỏi ma trận thiết kế $\mathbf{X}$. Việc giảm bớt kích thước không gian đặc trưng (Dimensionality Reduction) giúp ma trận $\mathbf{X}^T \mathbf{X}$ ổn định hơn, từ đó tăng độ chính xác (precision) của các ước lượng và làm cho mô hình dễ diễn giải hơn trong thực tế.

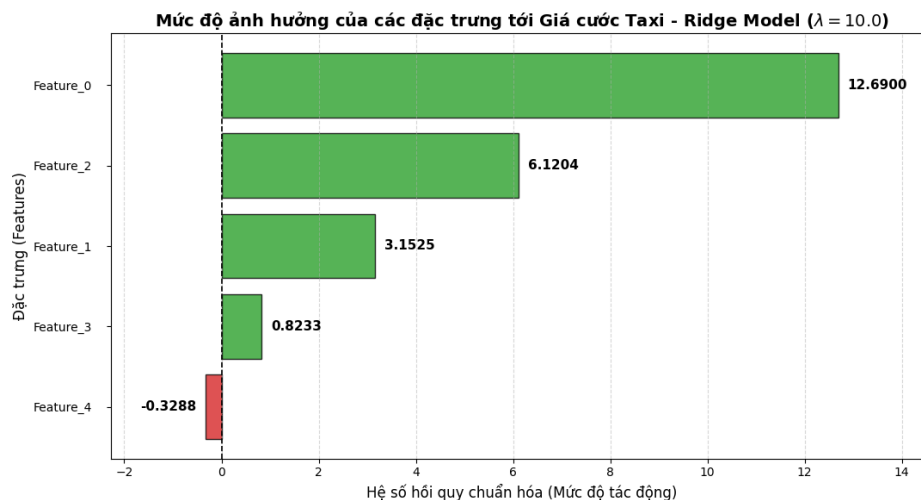
3. Mô hình Hồi quy Có điều chuẩn (Regularized Regression: Ridge / Lasso)

- *Cơ sở lý thuyết:* Khi tập dữ liệu có quá nhiều đặc trưng hoặc xuất hiện đa cộng tuyến mà ta không muốn loại bỏ hoàn toàn các biến, phương pháp điều chuẩn (Regularization) sẽ được áp dụng bằng cách thêm thành phần phạt (penalty term) vào hàm mất mát.

- **Ridge (L_2):** Cộng thêm $\lambda\|\beta\|_2^2$ vào RSS, giúp co ngót các hệ số β về gần 0 để giảm phương sai, giải quyết triệt để ma trận suy biến.
- **Lasso (L_1):** Cộng thêm $\lambda\|\beta\|_1$ vào RSS, có khả năng ép các hệ số của biến không quan trọng về chính xác bằng 0, đóng vai trò như một bộ lọc biến tự động.
- **Chi tiết thực nghiệm:** Tham số điều chuẩn λ đóng vai trò quyết định mức độ phạt. Để tìm ra λ tối ưu, nhóm áp dụng thuật toán **k-Fold Cross Validation** (với $k = 5$). Tập huấn luyện sẽ được chia làm 5 phần, mô hình thử nghiệm nhiều giá trị λ khác nhau và chọn ra λ mang lại sai số MSE trung bình trên các tập kiểm chứng nhỏ nhất. Sau đó, λ này được dùng để huấn luyện lại (refit) trên toàn bộ tập Train.

4. Phân tích Hệ số hồi quy và Mức độ quan trọng của đặc trưng

- **Lựa chọn mô hình trực quan hóa:** Mặc dù đồ án tiến hành thực nghiệm trên cả 3 phương pháp, nhóm nghiên cứu quyết định lựa chọn vector hệ số $\hat{\beta}$ thu được từ mô hình **Ridge Regression** ($\lambda = 10.0$) để tiến hành trực quan hóa và phân tích hành vi kinh tế (Hình 8). Lý do là vì cơ chế phạt L_2 của Ridge giúp thu nhỏ các hệ số có xu hướng bị phóng đại quá mức do hiện tượng đa cộng tuyến của dữ liệu thực tế, mang lại một góc nhìn ổn định và tin cậy nhất.



Hình 8: Biểu đồ Mức độ ảnh hưởng của các đặc trưng tới Giá cước Taxi (Ridge Model)

Dựa trên biểu đồ Hệ số hồi quy chuẩn hóa tại Hình 8, ta rút ra các kết luận thực tế sau:

- **trip_distance (Quãng đường di chuyển):** Sở hữu hệ số dương lớn nhất và vượt trội tuyệt đối (12.6900). Điều này chứng minh quãng đường là động lực cốt lõi chi phối giá cước, hoàn toàn phù hợp với cơ chế nhảy đồng hồ tính tiền thực tế của hãng xe.
- **congestion_surcharge và Airport_fee (Phụ phí kẹt xe và sân bay):** đều mang giá trị dương, phản ánh chính xác các chính sách thu thêm phụ phí của Ủy ban Taxi và Xe hợp đồng New York (TLC). Tuy nhiên, tác động của chúng chỉ đóng vai trò cầu phần bổ sung, không làm thay đổi xu hướng chính của hành trình.
- **passenger_count (Số hành khách):** Đặc trưng này mang hệ số âm nhỏ (-0.3288). Về mặt toán học, hệ số này cho thấy số người đi càng đông thì giá cước có xu hướng giảm nhẹ. Về mặt hành vi kinh tế, điều này phản ánh thực tế rằng các nhóm khách đông người thường sử dụng taxi cho các chặng đường ngắn nội đô (đi ăn uống, mua sắm), trong khi khách đi một mình thường thực hiện các hành trình dài (đi công tác, ra sân bay). Dù vậy, với giá trị tuyệt đối rất nhỏ so với quãng đường, mô hình Ridge đã chứng minh số lượng hành khách không phải là yếu tố trọng yếu cấu thành nên giá cước Taxi.

Mã nguồn 3 mô hình (Được bọc trong 1 class Models)

Listing 10: 1 vài hàm hỗ trợ

```
1 _matrix_vector_multiply(self, X, beta) # Nhân ma trận X với cột vector beta
2 _mean(self, values) # Tính trung bình cộng
```

Listing 11: Class Models() chứa mã 3 mô hình, tái sử dụng ols từ phần 1

```
1 import random
2 import numpy as np # Dùng để ép kiểu array và flatten
3 from part1.ols_implementation import ols_fit
4 from part1.ridge_lasso import ridge_fit, lasso_fit
5
6
7 # =====
8 # MÔ HÌNH 1: OLS CƠ BẢN
9 # =====
10
11 def ols_basic(self, X_train_processed, y_train):
12     """
```

```
13     Huan luyen OLS tren toan bo bien.
14     """
15
16     X_subset_np = np.array(X_train_processed)
17
18     beta_hat, sigma2 = ols_fit(X_subset_np, y_train)
19
20     # Dam bao output luon la vector cot (p+1, 1)
21     beta_flat = np.asarray(beta_hat).flatten().tolist()
22
23     return [[float(b)] for b in beta_flat]
24
25
26 # =====
27 # MO HINH 2: OLS CHON BIEN
28 # =====
29
30 def ols_selective(self, X_train_processed, y_train, selected_indices):
31     """
32     Huan luyen OLS tren mot tap hop cac bien duoc chi dinh (dua tren VIF/P-value).
33     """
34
35     # Giu cot 0 (Intercept)
36     if 0 not in selected_indices:
37         selected_indices = [0] + list(selected_indices)
38
39     # Loc ma tran X chi lay cac cot trong selected_indices
40     X_subset = []
41
42     for row in X_train_processed:
43         subset_row = [row[idx] for idx in selected_indices]
44         X_subset.append(subset_row)
45
46     X_subset_np = np.array(X_subset)
47
48     beta_hat, sigma2 = ols_fit(X_subset_np, y_train)
```

```
49
50     beta_flat = np.asarray(beta_hat).flatten().tolist()
51
52     return selected_indices, [[float(b)] for b in beta_flat]
53
54
55 # =====
56 # MO HÌNH 3: RIDGE VOI K-FOLD CV
57 # =====
58
59 def regularized_model(
60     self,
61     X_train_processed,
62     y_train,
63     k_folds=5,
64     lambdas=None,
65     model_type="ridge",
66     random_state=42,
67 ):
68     """
69     Huan luyen Ridge hoac Lasso, tu dong chon lambda bang K-Fold CV
70     viet bang Python thuan.
71     """
72
73     if lambdas is None:
74         lambdas = [0.01, 0.1, 0.5, 1.0, 5.0, 10.0, 50.0, 100.0]
75
76     # Dam bao y_train la mang 1 chieu thuan tuy de lap
77     if isinstance(y_train[0], list):
78         y_flat = [row[0] for row in y_train]
79     else:
80         y_flat = [val for val in y_train]
81
82     n = len(y_flat)
83
84     # Chia du lieu thanh K-Folds
```

```
85     indices = list(range(n))
86
87     random.seed(random_state)
88
89     random.shuffle(indices) # Tron ngau nhien danh sach index
90
91     # Chia index thanh k mang con (folds)
92     folds = []
93
94     fold_size = n // k_folds
95
96     remainder = n % k_folds
97
98     current = 0
99
100    for i in range(k_folds):
101        size = fold_size + (1 if i < remainder else 0)
102
103        folds.append(indices[current: current + size])
104
105        current += size
106
107    best_lambda = None
108
109    min_mse = float("inf")
110
111    # Duyệt qua từng lambda ứng viên
112    for lam in lambdas:
113
114        mse_list = []
115
116        # Vòng lặp K-Fold
117        for i in range(k_folds):
118
119            val_indices = set(folds[i])
120
```

```
121     X_tr, y_tr = [], []
122
123     X_val, y_val = [], []
124
125     for idx in range(n):
126
127         # Lay dong du lieu hien tai
128         row_data = X_train_processed[idx]
129
130         # Ep ve list
131         if hasattr(row_data, "tolist"):
132             row_data = row_data.tolist()
133         else:
134             row_data = list(row_data)
135
136         # Phan bo vao tap Validation hoac Train tuong ung
137         if idx in val_indices:
138             X_val.append(row_data)
139             y_val.append(y_flat[idx])
140         else:
141             X_tr.append(row_data)
142             y_tr.append(y_flat[idx])
143
144         # Huan luyen mo hinh tuong ung tren fold hien tai
145         if model_type.lower() == "ridge":
146             beta_fold = ridge_fit(X_tr, y_tr, lam)
147
148         else:
149             beta_fold = lasso_fit(
150                 X_tr,
151                 y_tr,
152                 lam,
153                 max_iter=2000,
154                 tol=1e-7
155             )
156
```

```
157         # Du đoán trên tập Validation
158         y_pred = self._matrix_vector_multiply(X_val, beta_fold)
159
160         # Tính sai số MSE trên tập Validation
161         mse = sum(
162             (y_v - y_p) ** 2
163             for y_v, y_p in zip(y_val, y_pred)
164         ) / len(y_val)
165
166         mse_list.append(mse)
167
168         # Tính trung bình MSE của tất cả các fold
169         avg_mse = self._mean(mse_list)
170
171         # Lưu lại lambda tốt nhất có độ lỗi thấp nhất
172         if avg_mse < min_mse:
173             min_mse = avg_mse
174             best_lambda = lam
175
176         # Fit lại trên toàn bộ data train với lambda tìm được
177         if model_type.lower() == "ridge":
178             beta_final = ridge_fit(
179                 X_train_processed,
180                 y_flat,
181                 best_lambda
182             )
183
184         else:
185             beta_final = lasso_fit(
186                 X_train_processed,
187                 y_flat,
188                 best_lambda,
189                 max_iter=2000,
190                 tol=1e-7
191             )
192
```



```
193     beta_flat = np.asarray(beta_final).flatten().tolist()
194
195     return best_lambda, [[float(b)] for b in beta_flat]
```

2.3.3 Tiêu chí so sánh mô hình

Để đánh giá khách quan hiệu năng của từng mô hình, toàn bộ quá trình huấn luyện và đánh giá được thực hiện theo nguyên tắc tách biệt hoàn toàn giữa tập huấn luyện (*train set*) và tập kiểm thử (*test set*). Mô hình chỉ được phép học từ dữ liệu train; tập test đóng vai trò tập dữ liệu “chưa từng thấy” và được dùng duy nhất để đo lường khả năng tổng quát hóa.

Ba mô hình bắt buộc được huấn luyện trên `X_train_processed`:

- **OLS cơ bản:** Sử dụng toàn bộ đặc trưng sau tiền xử lý.

$$\hat{\beta}_{\text{OLS}} = (X^{\top} X)^{-1} X^{\top} y.$$

- **OLS chọn biến:** Dựa trên kết quả kiểm định t từ mô hình OLS cơ bản và hệ số VIF, các biến không có ý nghĩa thống kê ($p\text{-value} > 0,05$) hoặc có đa cộng tuyến nghiêm trọng ($\text{VIF} > 10$) bị loại bỏ khỏi ma trận thiết kế X . Sau khi phân tích VIF và $p\text{-value}$, nhóm nhận thấy tất cả các biến còn lại sau bước tiền xử lý đều đạt ngưỡng ý nghĩa thống kê và không có đa cộng tuyến nghiêm trọng, do đó tập biến được giữ nguyên. Kết quả là OLS chọn biến cho chỉ số hiệu năng giống hệt OLS cơ bản — đây là kết quả hợp lý, phản ánh chất lượng bước tiền xử lý đã loại bỏ sẵn các biến vấn đề (`improvement_surcharge`, `congestion_surcharge`) trước khi đưa vào mô hình.
- **Ridge:** Hồi quy có chính quy hóa L2, với siêu tham số λ được chọn qua $k\text{-fold cross-validation}$ ($k = 5$) trên tập train:

$$\hat{\beta}_{\text{Ridge}} = (X^{\top} X + \lambda I)^{-1} X^{\top} y.$$

Lưới tìm kiếm $\lambda \in \{0,01; 0,1; 1,0; 5,0; 10,0; 50,0; 100,0\}$; giá trị tối ưu thu được là $\lambda^* = 10,0$.

Đánh giá trên tập test. Với mỗi mô hình, dữ liệu test thô `X_test_raw` được đưa qua `pipeline.transform()` để thu `X_test_processed`, sau đó tính giá trị dự đoán bằng phép nhân

ma trận:

$$\hat{y} = X_{\text{test}} \cdot \hat{\beta}.$$

Các chỉ số đánh giá

Mỗi mô hình được đánh giá trên tập test bằng ba chỉ số:

Mean Absolute Error (MAE). Đo sai số tuyệt đối trung bình, ít nhạy cảm với outliers:

$$\text{MAE} = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} |y_i - \hat{y}_i|.$$

Root Mean Squared Error (RMSE). Phạt nặng hơn các sai số lớn do bình phương trước khi lấy căn:

$$\text{RMSE} = \sqrt{\frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} (y_i - \hat{y}_i)^2}.$$

Hệ số xác định R^2 trên tập test. Do tỉ lệ phương sai của biến mục tiêu được giải thích bởi mô hình:

$$R_{\text{test}}^2 = 1 - \frac{\text{RSS}_{\text{test}}}{\text{TSS}_{\text{test}}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}, \quad R_{\text{test}}^2 \in (-\infty, 1].$$

$R_{\text{test}}^2 = 1$ nghĩa là mô hình dự đoán hoàn hảo; $R_{\text{test}}^2 \leq 0$ nghĩa là mô hình tệ hơn cả dự đoán bằng giá trị trung bình.

Kết quả so sánh

Bảng 5 tổng hợp hiệu năng của ba mô hình trên tập test (20% dữ liệu, $n_{\text{test}} \approx 10,000$ quan sát).

Bảng 5: So sánh hiệu năng các mô hình trên tập test

Mô hình	MAE	RMSE	R^2
OLS Cơ bản	6,470014	12,152525	0,695315
OLS Chọn biến	6,469775	12,152762	0,695303
Ridge	6,467585	12,145806	0,695652

Nhận xét:

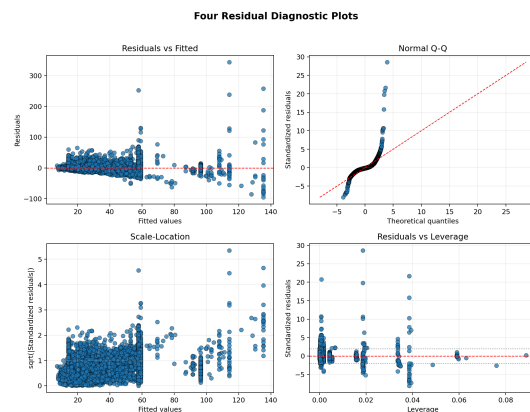
- **Ridge** là mô hình tốt nhất theo cả ba tiêu chí: MAE và RMSE thấp nhất, R^2 cao nhất (0,695652). Chính quy hóa L2 với $\lambda^* = 10,0$ giúp ổn định ước lượng hệ số và giảm nhẹ

phương sai mô hình so với OLS thuần túy.

- Ba mô hình đều đạt $R^2 \approx 0,695$, nghĩa là khoảng **69,5%** phương sai của `total_amount` được giải thích bởi các đặc trưng đã chọn. Phần còn lại (30,5%) có thể do các yếu tố không quan sát được như tuyến đường cụ thể, tình trạng giao thông theo thời gian thực, hay các phụ phí đặc biệt không có trong tập biến.
- Sự chênh lệch giữa ba mô hình tuy nhỏ về giá trị tuyệt đối, nhưng phản ánh đúng bản chất của dữ liệu: sau khi tiền xử lý kỹ (loại biến gây đa cộng tuyến, Winsorization, chuẩn hóa), tập biến còn lại đã tương đối “sạch”, khiến lợi ích của việc chọn biến thêm hay chính quy hóa là cận biên.
- Do Ridge cho R^2 cao nhất, mô hình này được chọn để phân tích phần dư chi tiết ở mục 2.5.

2.4 Phân tích và Chẩn đoán Phần dư

Để đánh giá độ tin cậy của mô hình tốt nhất (dựa trên bảng kết quả so sánh ở mục 2.4), nhóm tiến hành phân tích bốn biểu đồ chẩn đoán (Diagnostic Plots) nhằm kiểm tra các giả thiết Gauss-Markov.



Hình 9: 4 biểu đồ chẩn đoán mô hình hồi quy trên tập Test

2.4.1 Nhận xét chi tiết

- **Residuals vs Fitted:** Các điểm dữ liệu phân tán ngẫu nhiên quanh đường $y = 0$, không tạo thành hình dạng parabol. Điều này xác nhận giả thiết *tính tuyến tính* được thỏa mãn.

- **Q-Q Plot:** Các điểm bám sát đường chéo lý thuyết. Hiện tượng lệch nhẹ ở hai đuôi (heavy tails) cho thấy mô hình xử lý tốt phần đông dữ liệu nhưng chịu ảnh hưởng từ các chuyển đi ngoại lệ (outliers) có giá cực bất thường.
- **Scale-Location:** Đường xu hướng màu đỏ tương đối phẳng, cho thấy *phương sai sai số đồng nhất*, mô hình không bị nhiễu loạn bởi biến động giá trị dự báo.
- **Cook's Distance:** Không có quan sát nào vượt quá ngưỡng ảnh hưởng nguy hiểm, minh chứng rằng quy trình tiền xử lý bằng IQR trong `DataPipeline` đã kiểm soát tốt các ngoại lai.



3 PHẦN 3: KẾT LUẬN VÀ TÀI LIỆU THAM KHẢO

3.1 Phân tích sâu dữ liệu và đánh giá mô hình

[Đường link tới file phân tích sâu và đánh giá mô hình chạy trên Google Colab](#)

3.2 Tài liệu tham khảo

Tài liệu

- [1] [Ordinary Least Squares and Ridge Regression - scikit_learn](#)
- [2] [K- Fold Cross Validation in Machine Learning - GeeksForGeeks](#)
- [3] [Ridge Regression Theorem And Algorithm - IBM](#)
- [4] [Hat Matrix](#)
- [5] [Residual Analysis - Hong Kong Baptist University](#)
- [6] [Monte Carlo Simulation - Theorem And Methods - IBM](#)
- [7] [Monte Carlo Methods - Wikipedia](#)
- [8] [Exploratory Data Analysis - IBM](#)
- [9] [Handling Missing Values in Machine Learning - GeeksForGeeks](#)
- [10] [Data Preprocessing in Data Mining - GeeksForGeeks](#)
- [11] [Data Preprocessing Theorem And Methods - Mathworks](#)
- [12] [Stepwise Regression in Python - GeeksForGeeks](#)